

第四章

进程管理

主要内容

2.1 进程及其调度控制与死锁

2.2 UNIX的进程

2.3 中断的基本概念及UNIX中断处理

2.4 UNIX时钟中断与系统调用

2.5 UNIX的进程调度状态

2.6 进程通讯

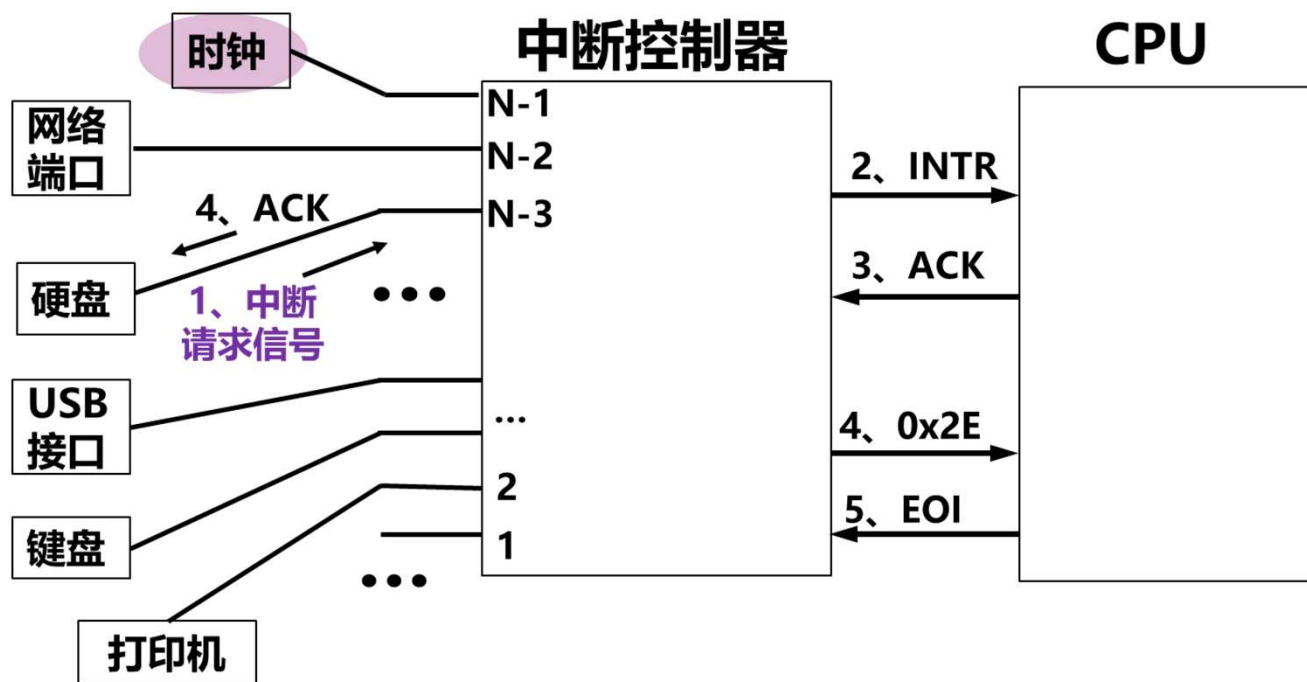
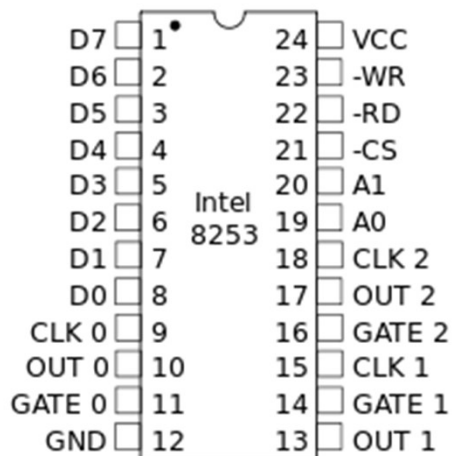


UNIX时钟中断



PC中的可编程定时芯片：8253或8254
每秒60次定时中断：“心跳”

时钟中断的中
断号为：0x20
优先级最高



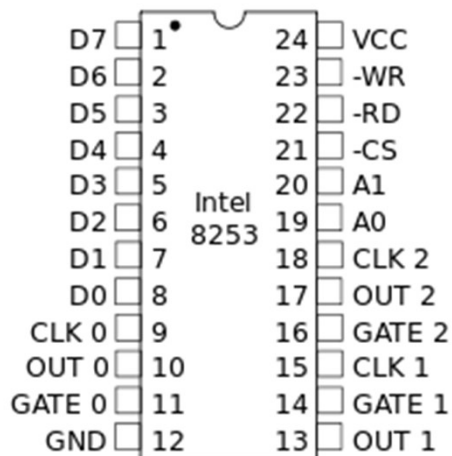


UNIX时钟中断



PC中的可编程定时芯片：8253或8254
每秒60次定时中断：“心跳”

时钟中断的中
断号为：0x20
优先级最高



class Time

{
public:

static const int SCHMAG = 10 * 2;

/* 每秒钟减少的p_cpu魔数 */

static const int HZ = 60;

/* 每秒钟时钟中断次数 */

static int lbolt;

/* 累计接收到的时钟中断次数 */

static unsigned int time; /* 系统全局时间，自1970年1月1日至今的秒数 */

static unsigned int tout; /* 各延时睡眠进程中应被唤醒的时刻中最小值 */

/*时钟中断入口程序，其地址存放在IDT的时钟中断对应中断门中 */

static void TimeInterruptEntrance();

/* 时钟中断处理程序 */

static void Clock(struct pt_regs* regs, struct pt_context* context);

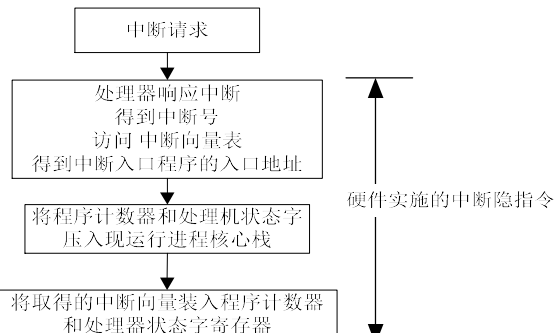
};



UNIX时钟中断



硬件实施中断隐指令



时钟中断中处理所有和时间相关的操作

```
void Time::TimeInterruptEntrance()
{
    /* 保存中断现场 */
    SaveContext();

    /* 完全进入核心态 */
    SwitchToKernel();

    /* 调用时钟中断处理程序 */
    CallHandler(Time, Clock);

    /* 例行调度 */
    .....

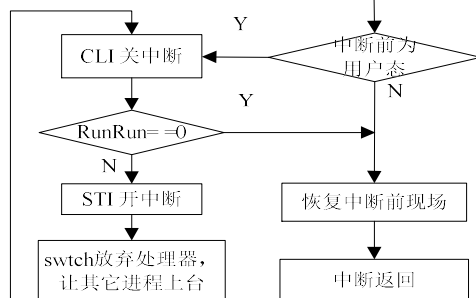
    /* 恢复现场 */
    RestoreContext();
    /* 手工销毁栈帧 */
    Leave();
    /* 退出中断 */
    InterruptReturn();
}
```

概况

时钟中断入口程序 Time::TimeInterruptEntrance

时钟中断处理程序 Time::Clock

EOI调整位置





UNIX中断处理流程



UNIX V6++定义的中断入口程序

中断号/中断源		中断入口程序	中断处理子程序
0x0	除0错	Exception::DivideErrorEntrance()	Exception::DivideError (struct pt_regs* regs, struct pt_context* context);
0x1	调试异常	Exception::DebugEntrance();	Exception::Debug (struct pt_regs* regs, struct pt_context* context);
0x2	NMI非屏蔽中断	Exception::NMIEntrance();	Exception:: NMI (struct pt_regs* regs, struct pt_context* context);
0x3	调试断点	Exception::BreakpointEntrance();	Exception:: Breakpoint (struct pt_regs* regs, struct pt_context* context);
.....			
0x1F	保留异常		
0x20	时钟中断	Time::TimeInterruptEntrance()	void Time::Clock(struct pt_regs* regs, struct pt_context* context)
0x21	键盘中断	KeyboardInterrupt::KeyboardInterruptEntrance()	void Keyboard::KeyboardHandler(struct pt_regs* reg, struct pt_context* context)
...	...		
0x2E	硬盘中断	void DiskInterrupt::DiskInterruptEntrance()	void ATADriver::ATAHandler(struct pt_regs *reg, struct pt_context* context)
....	...	...	
0x80	系统调用	void SystemCall::SystemCallEntrance()	void SystemCall::Trap(struct pt_regs* regs, struct pt_context* context)
...	...	...	...

在中断描述符表的0x20号中断处登记地址

入口程序跳转至处理程序



UNIX时钟中断



概况

	名称	类型	含义
进程标识	p_uid	short	用户ID
	p_pid	int	进程标识数, 进程编号
	p_ppid	int	父进程标识数
进程图象在内存中的位置信息	p_addr	unsigned long	ppda区在物理内存中的起始地址
	p_size	unsigned int	进程图象 (除代码段以外部分) 的长度, 以字节单位
	p_textp	Text *	指向该进程所运行的代码段的描述符
进程调度相关信息	p_stat	ProcessState	进程当前的调度状态
	p_flag	int	进程标志位, 可以将多个状态组合
	p_pri	int	进程优先数 (值越大, 优先级越小)
	p_cpu	int	cpu值, 用于计算p_pri
	p_nice	int	进程优先数微调参数
	p_time	int	进程在盘交换区上 (或内存内) 的驻留时间
	p_wchan	unsigned long	进程睡眠原因
信号与控制台终端	p_sig	int	进程信号
	p_ttyp	TTy*	进程tty结构地址



UNIX时钟中断



概况

	名称	类型	含义
进程标识	p_uid	short	用户ID
	p_pid	int	进程标识数, 进程编号
	p_ppid	int	父进程标识数
进程图象在内存中的位置信息	p_addr	unsigned long	ppda区在物理内存中的起始地址
	p_size	unsigned int	进程图象 (除代码段以外部分) 的长度, 以字节单位
	p_textp	Text *	指向该进程所运行的代码段的描述符
进程调度相关信息	p_stat	ProcessState	进程当前的调度状态
	p_flag	int	进程标志位, 可以将多个状态组合
	p_pri	int	进程优先数 (值越大, 优先级越小)
	p_cpu	int	时间每过去1秒, p_time++; 图像每交换一次, p_time = 0
	p_nice	int	
	p_time	int	进程在盘交换区上 (或内存内) 的驻留时间
	p_wchan	unsigned long	进程睡眠原因
信号与控制台终端	p_sig	int	进程信号
	p_ttyp	TTy*	进程tty结构地址



UNIX时钟中断



概况

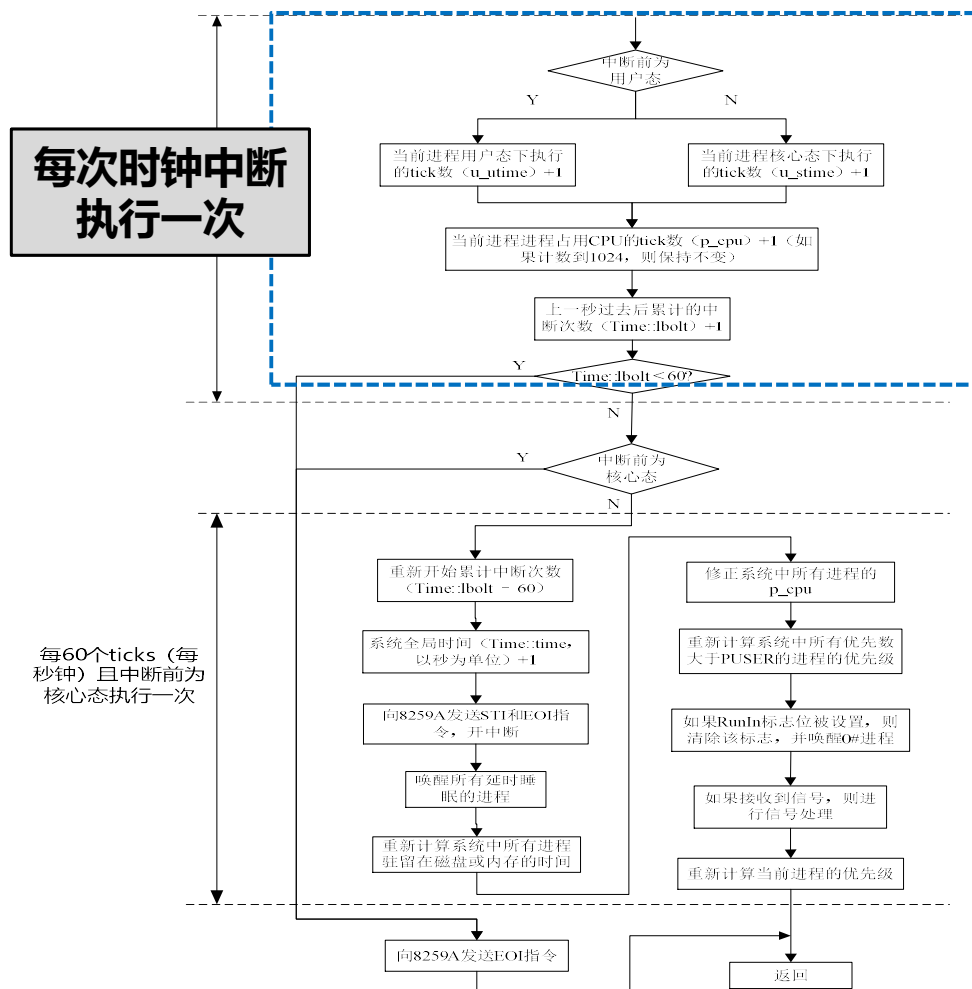
	名称	类型	含义
进程的用户标识	u_uid	short	有效用户ID
	u_gid	short	有效组ID
	u_ruid	short	真实用户ID
	u_rgid	short	真实组ID
进程的时间相关	u_utime	int	进程用户态时间
	u_stime	int	进程核心态时间
	u_cutime	int	子进程用户态时间总和
	u_cstime	int	子进程核心态时间总和
现场保护相关	u_rsav[2]	unsigned long	用于保存esp与ebp指针
	u_ssav[2]	unsigned long	用于对esp和ebp指针的二次保护
内存管理相关	*u_procp	Process	指向该u结构对应的Process结构
	u_MemoryDescriptor	MemoryDescriptor	封装了进程的图象在内存中的位置、大小等信息
出错	u_error	ErrorCode	存放错误码，具体数值及其含义请查阅源代码



UNIX时钟中断



时钟中断处理程序的详细流程

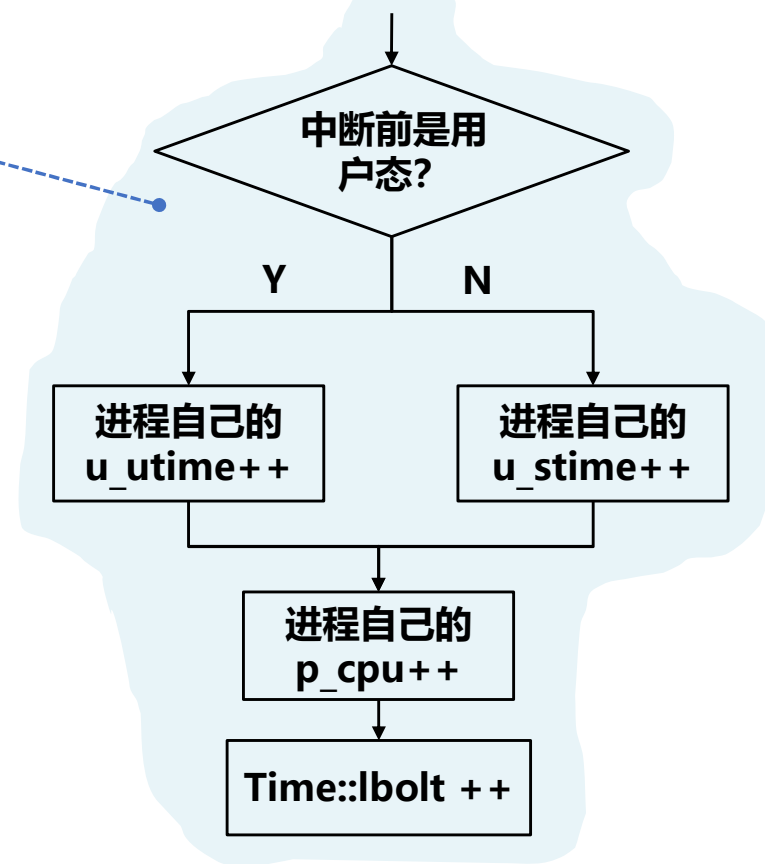
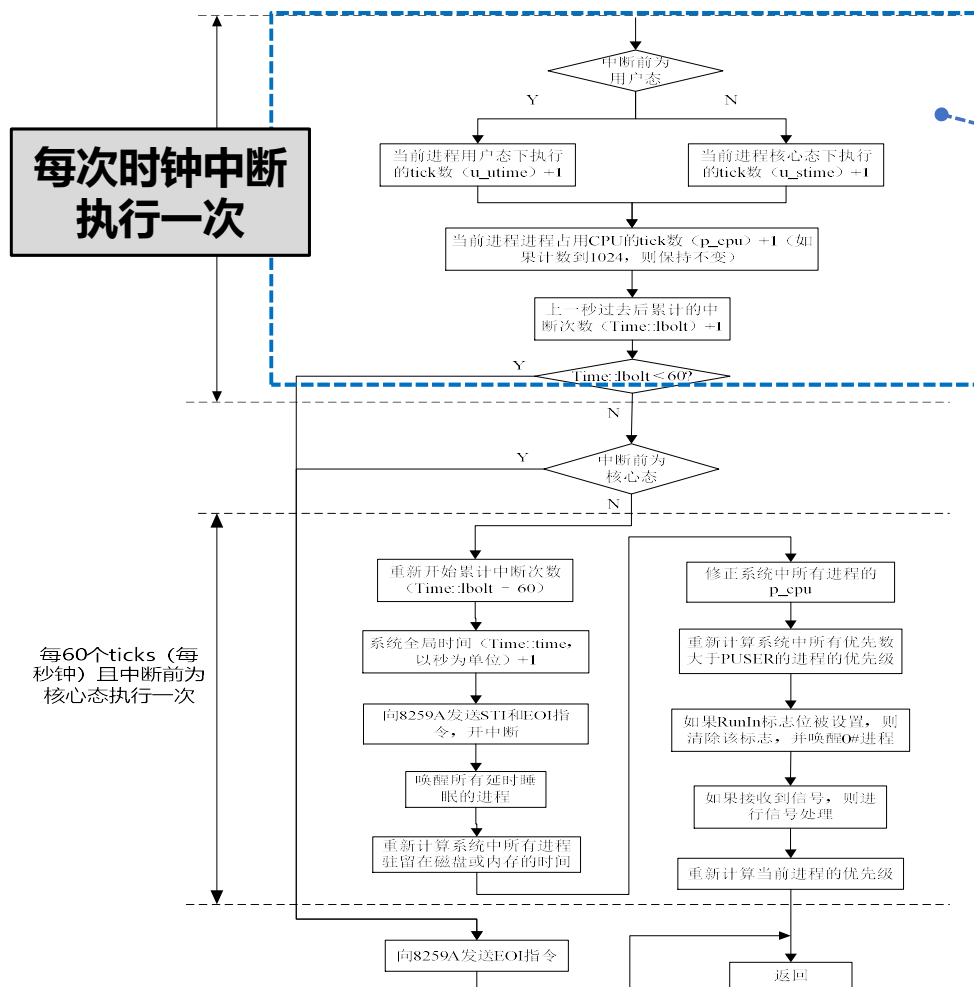




UNIX时钟中断



时钟中断处理程序的详细流程

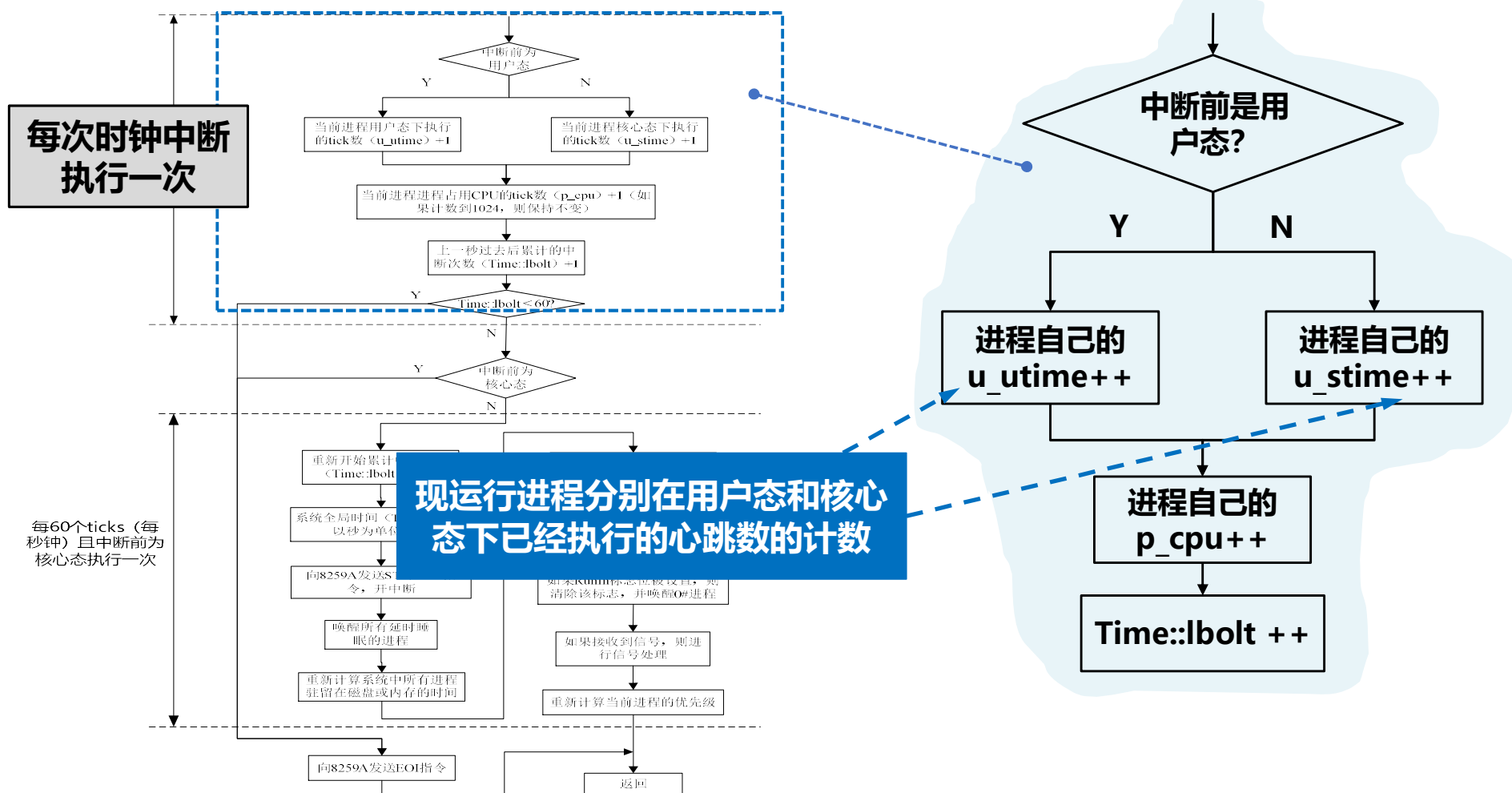




UNIX时钟中断



时钟中断处理程序的详细流程

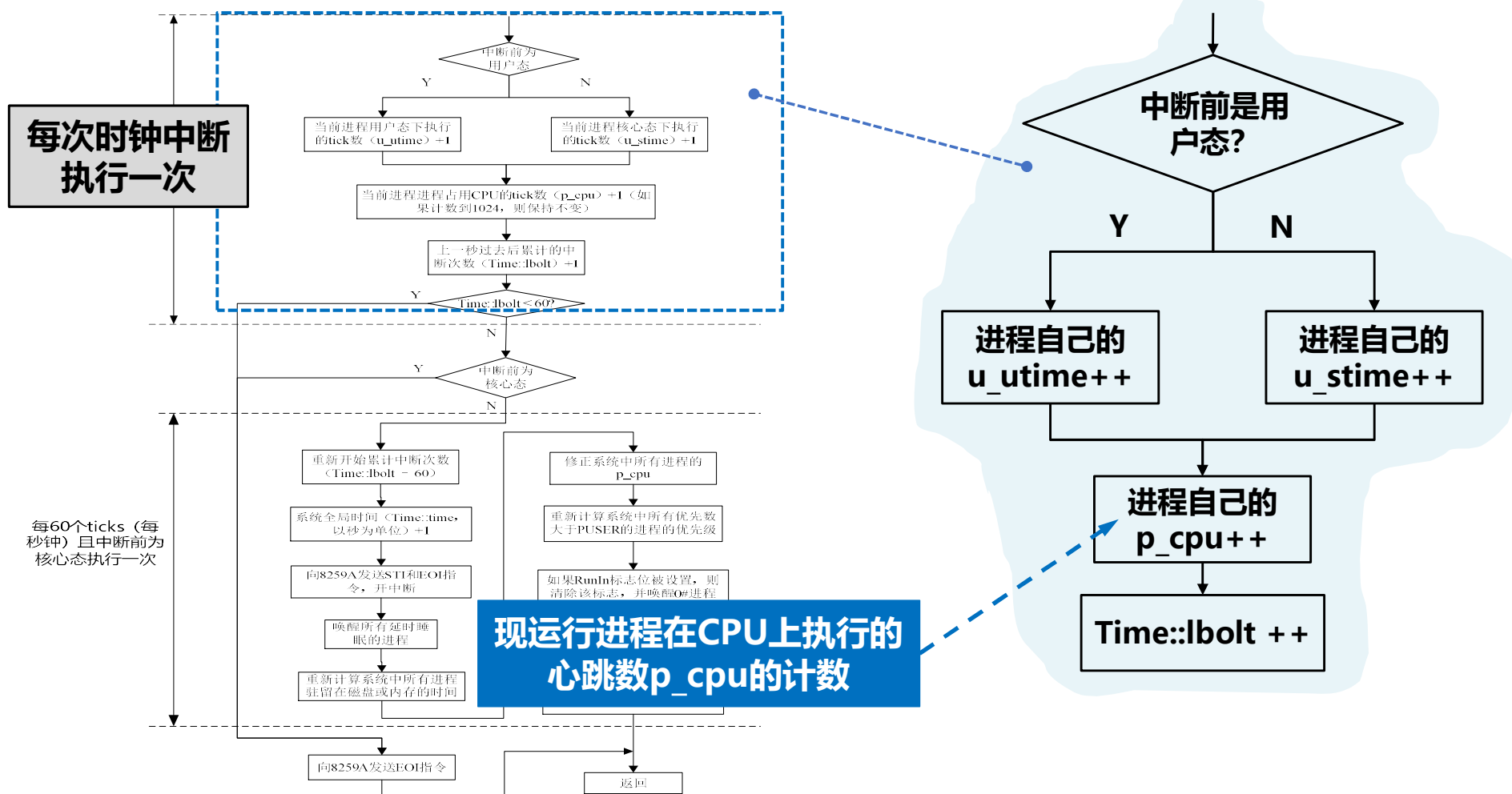




UNIX时钟中断



时钟中断处理程序的详细流程

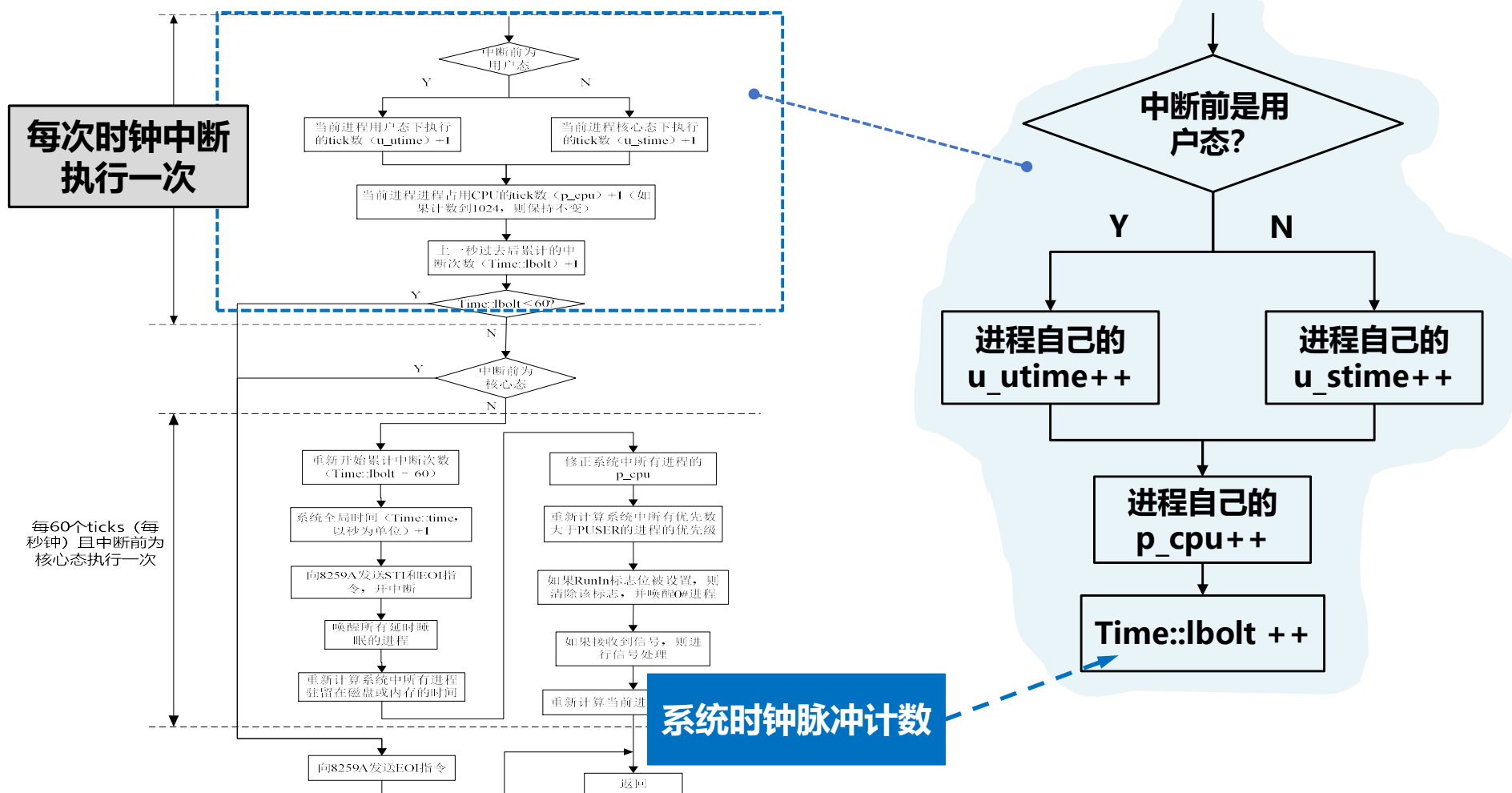




UNIX时钟中断



时钟中断处理程序的详细流程

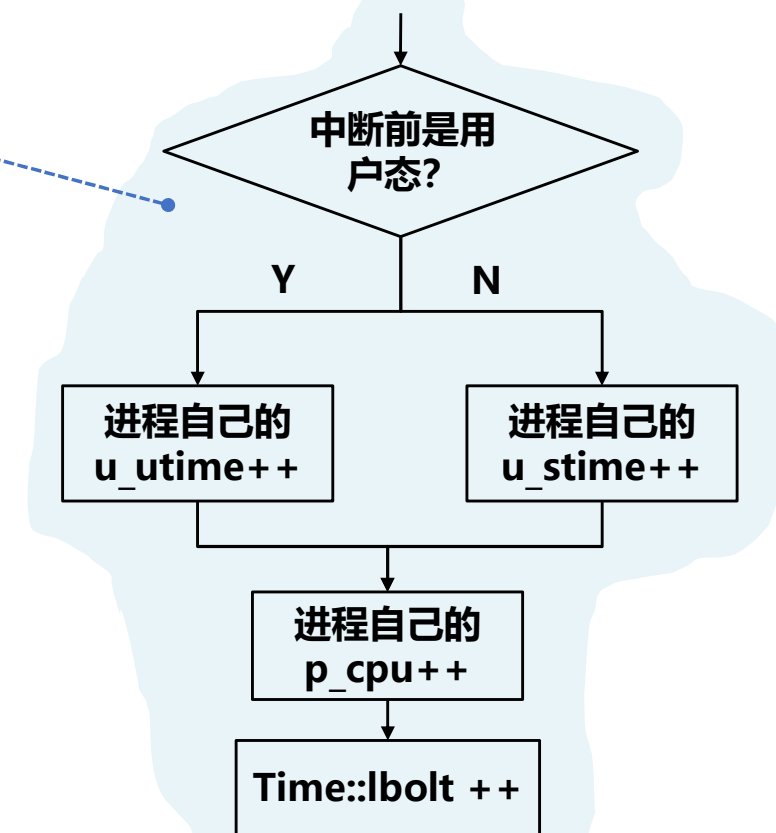
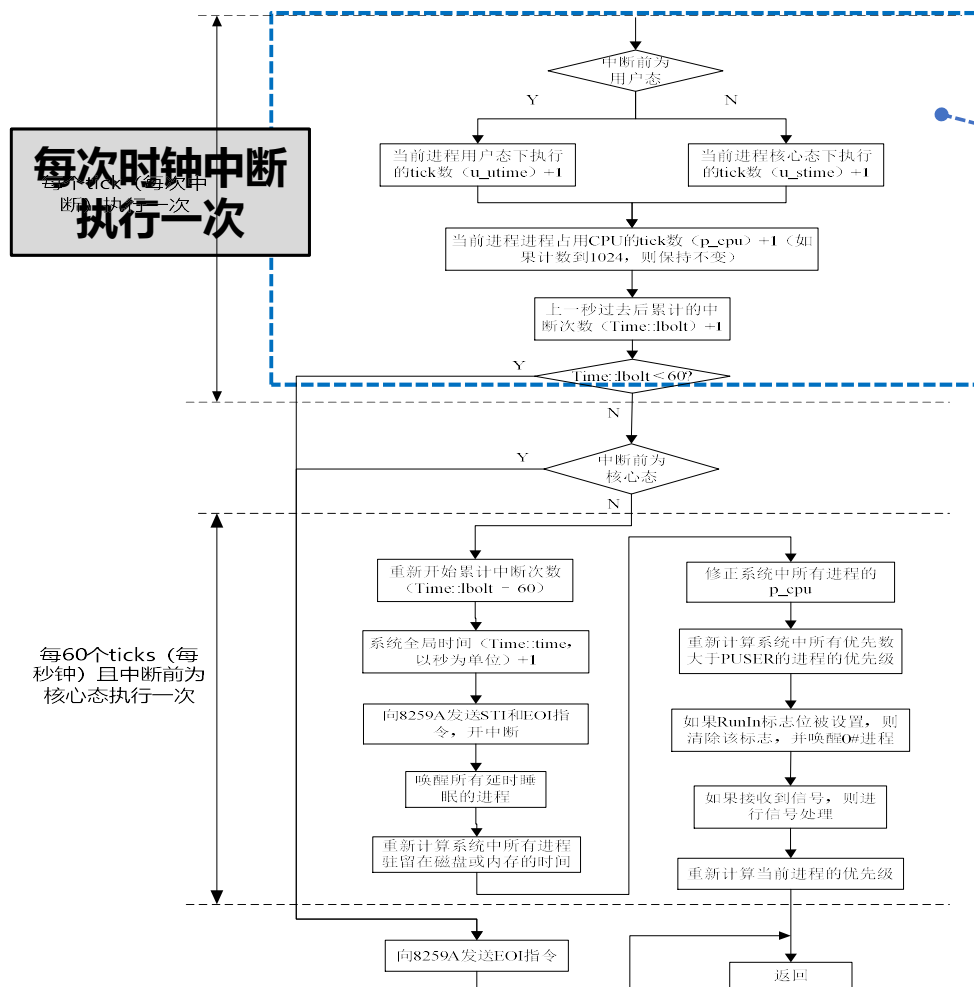




UNIX时钟中断



时钟中断处理程序的详细流程



所以，这些参数都是以时钟脉冲数为单位的

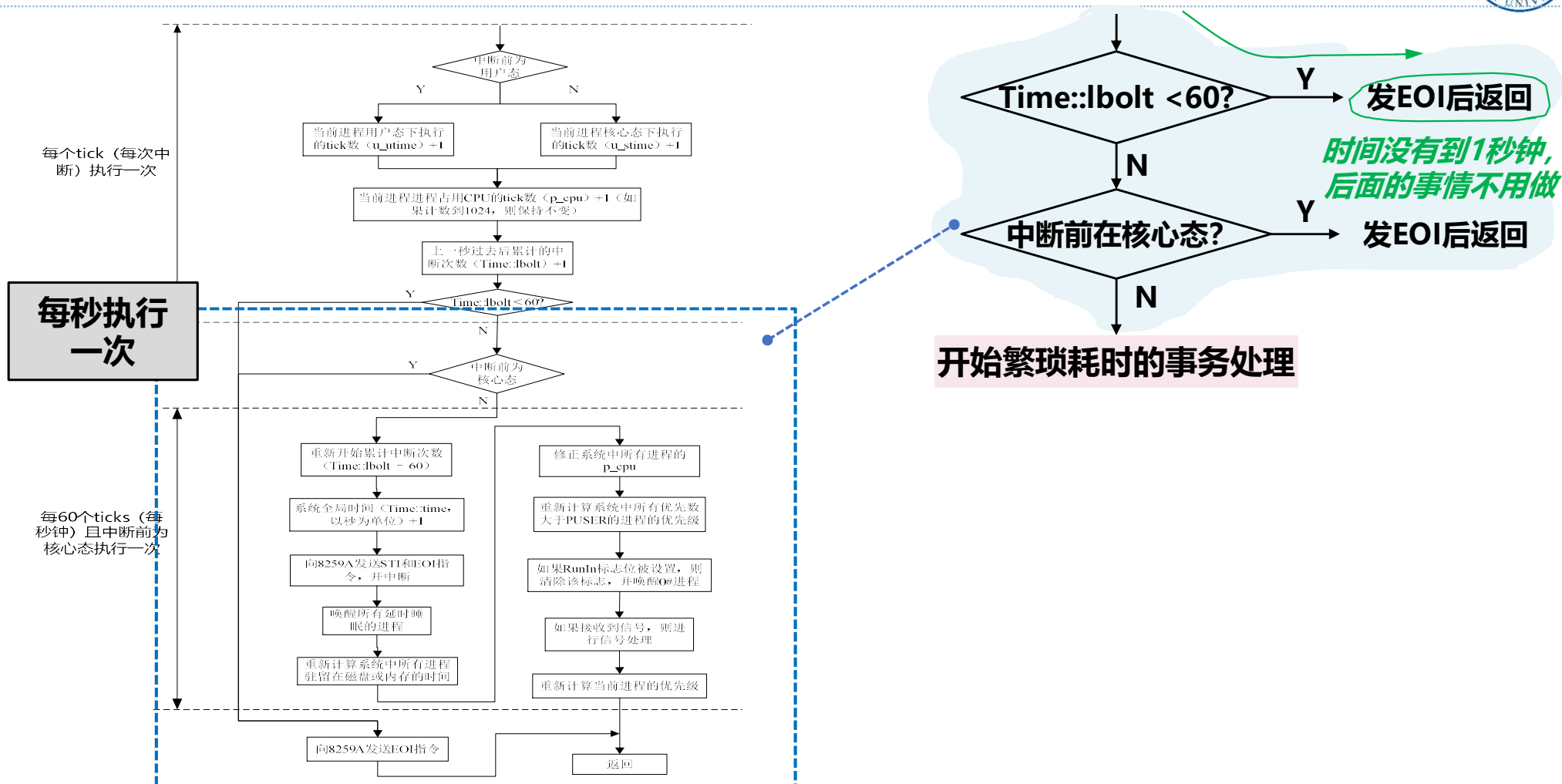




UNIX时钟中断



时钟中断处理程序的详细流程

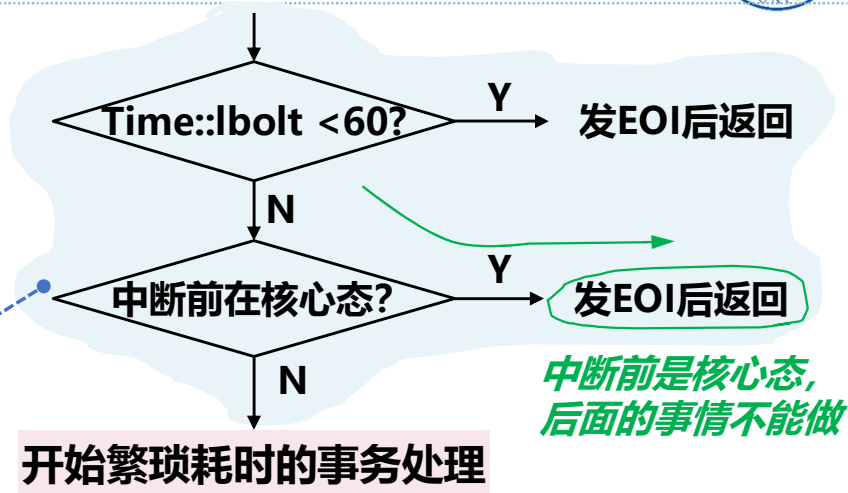
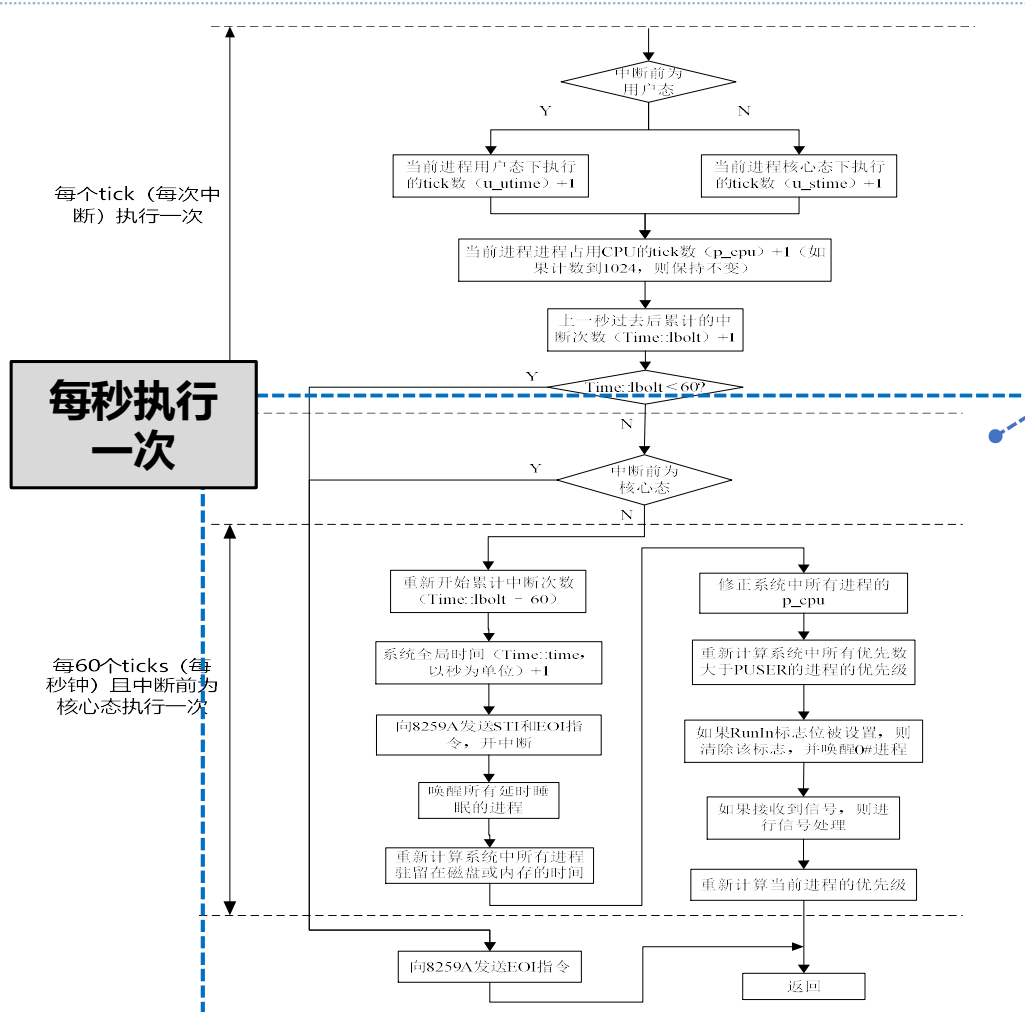




UNIX时钟中断



时钟中断处理程序的详细流程



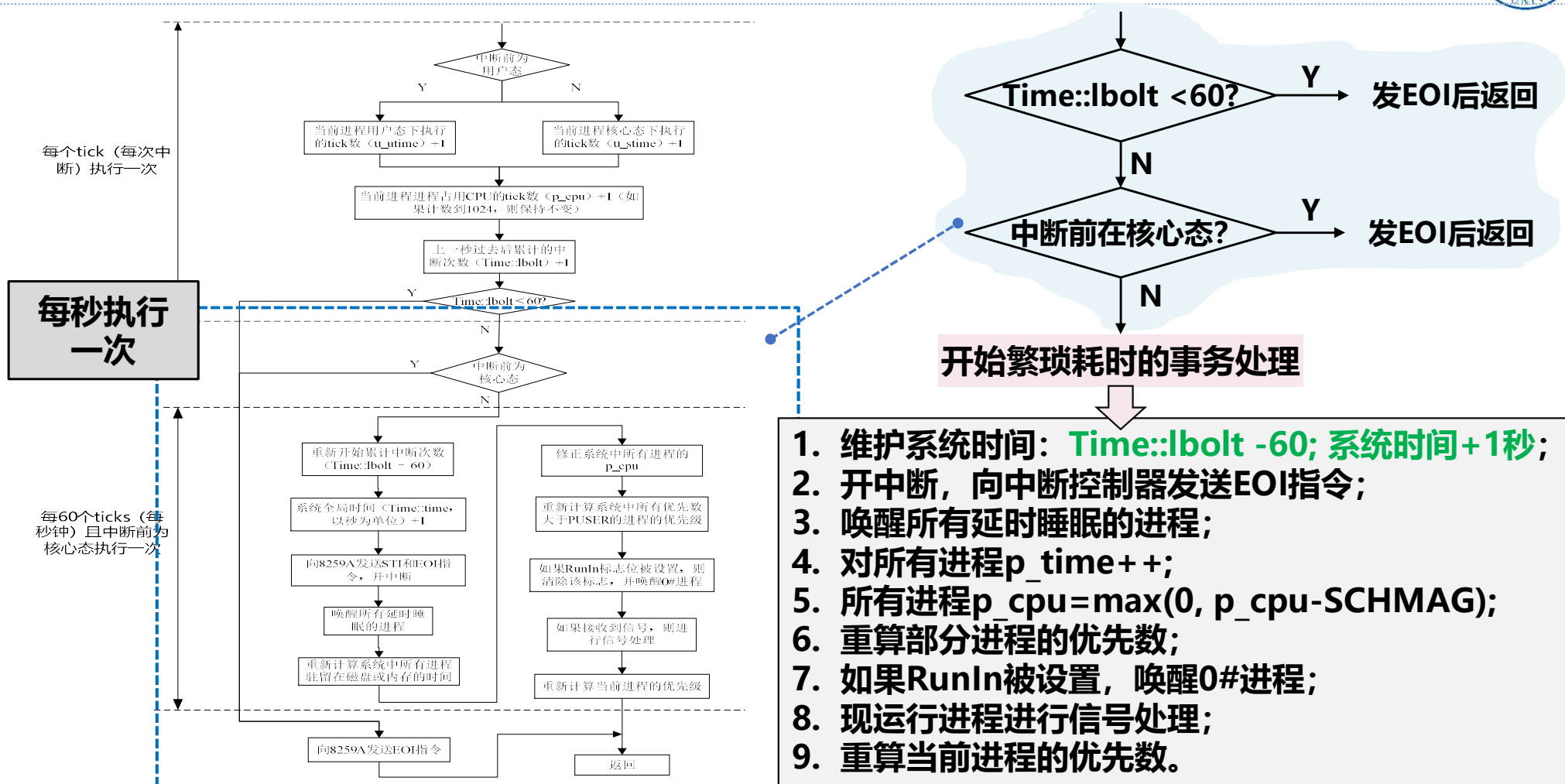
系统不会因为后续冗长的维护工作耽误核心态任务!!!



UNIX时钟中断



时钟中断处理程序的详细流程

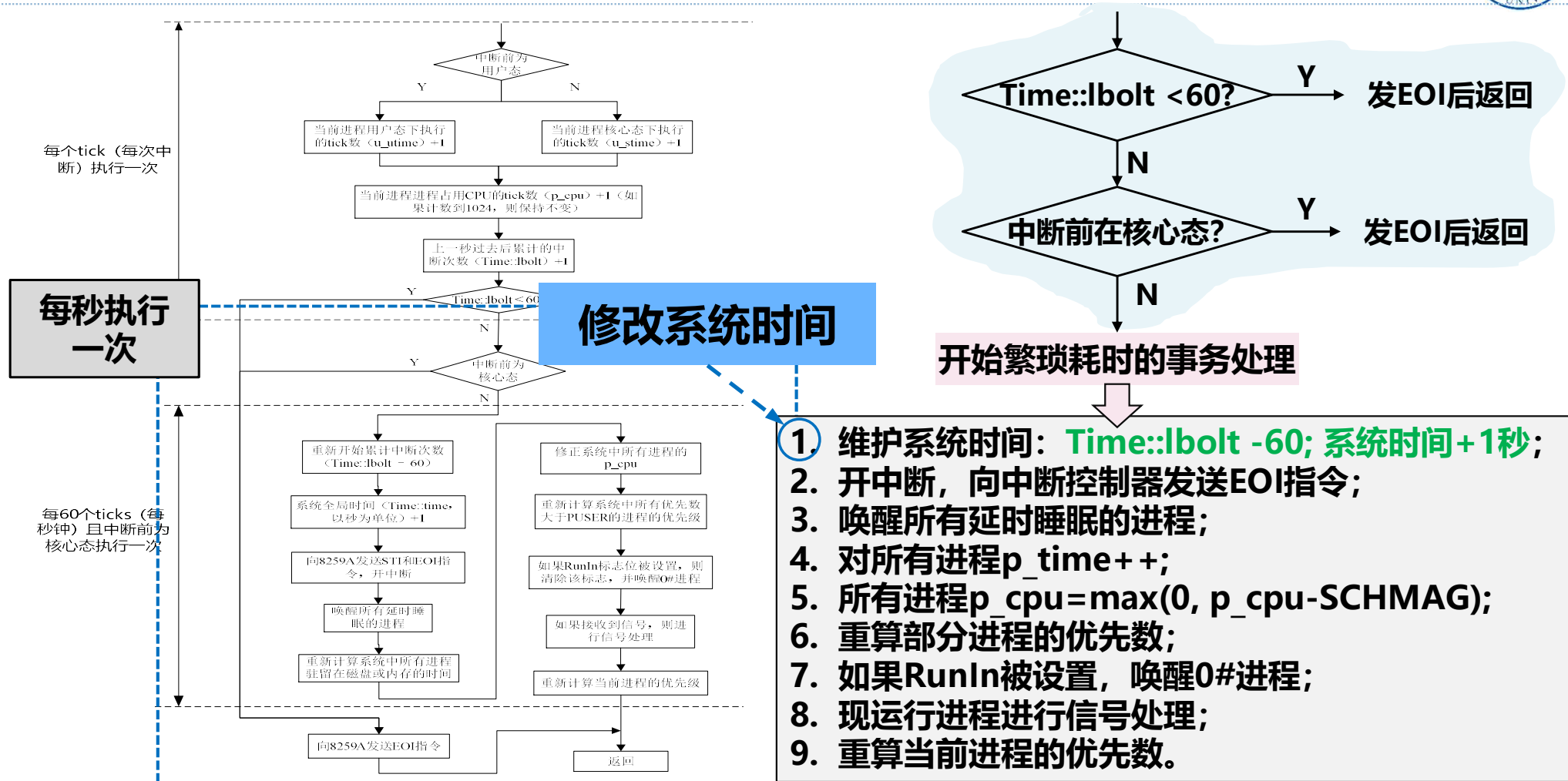




UNIX时钟中断



时钟中断处理程序的详细流程



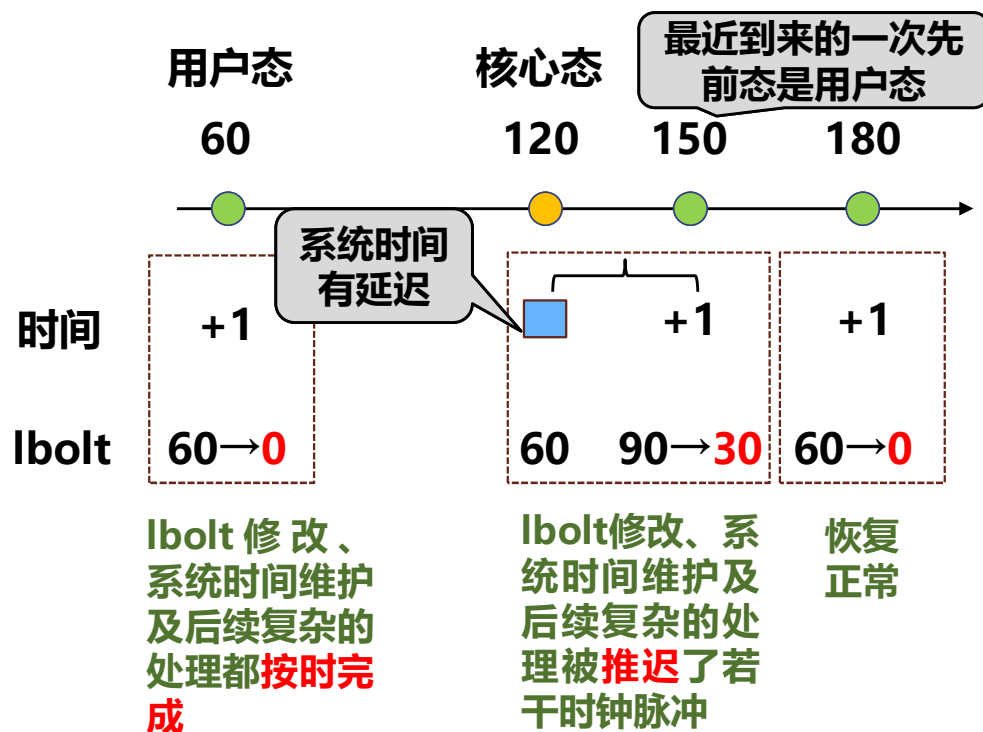


UNIX时钟中断

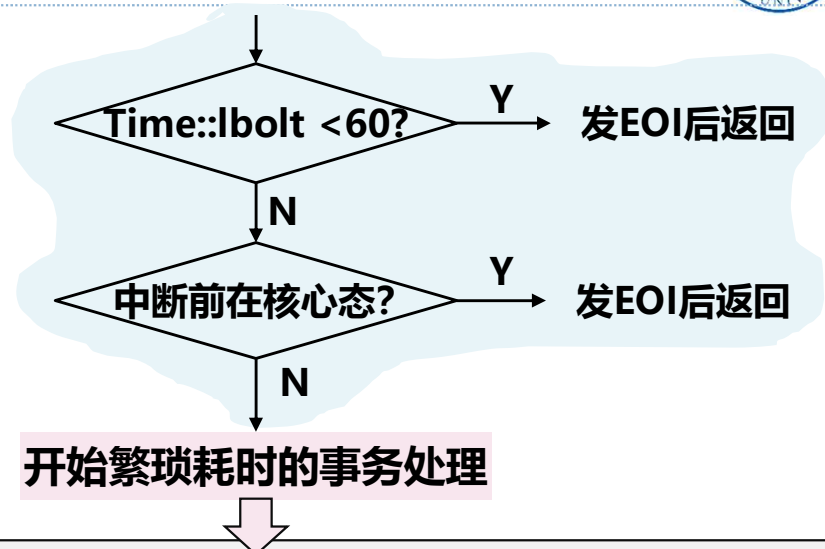


时钟中断处理程序的详细流程

如果整数秒的时钟中断先前态是核心态，是否会漏掉这一秒计时？



不漏掉一次时钟脉冲，但不急于修改系统时间



1. 维护系统时间: **Time::lbolt - 60; 系统时间 + 1秒;**
2. 开中断, 向中断控制器发送EOI指令;
3. 唤醒所有延时睡眠的进程;
4. 对所有进程 p_time++ ;
5. 所有进程 $p_cpu = \max(0, p_cpu - SCHMAG)$;
6. 重算部分进程的优先数;
7. 如果RunIn被设置, 唤醒0#进程;
8. 现运行进程进行信号处理;
9. 重算当前进程的优先数.

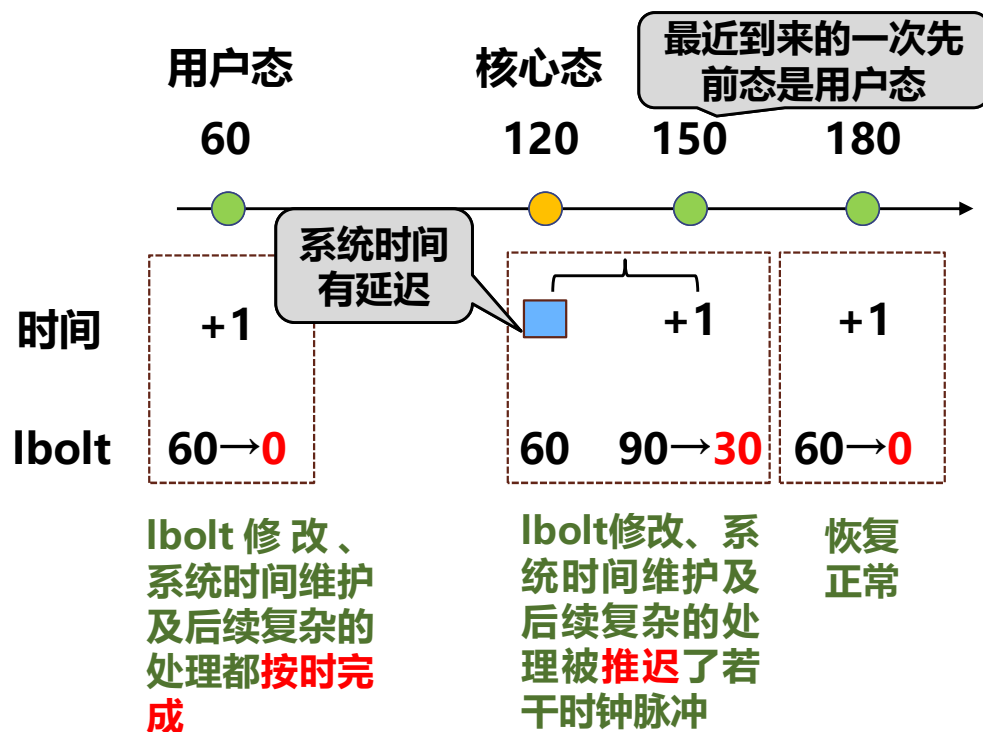


UNIX时钟中断

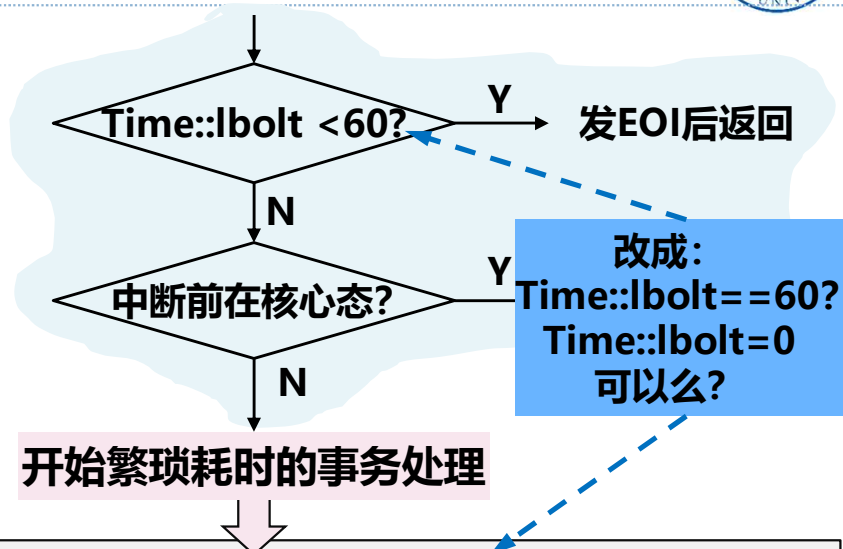


时钟中断处理程序的详细流程

如果整数秒的时钟中断先前态是核心态，是否会漏掉这一秒计时？



不漏掉一次时钟脉冲，但不急于修改系统时间



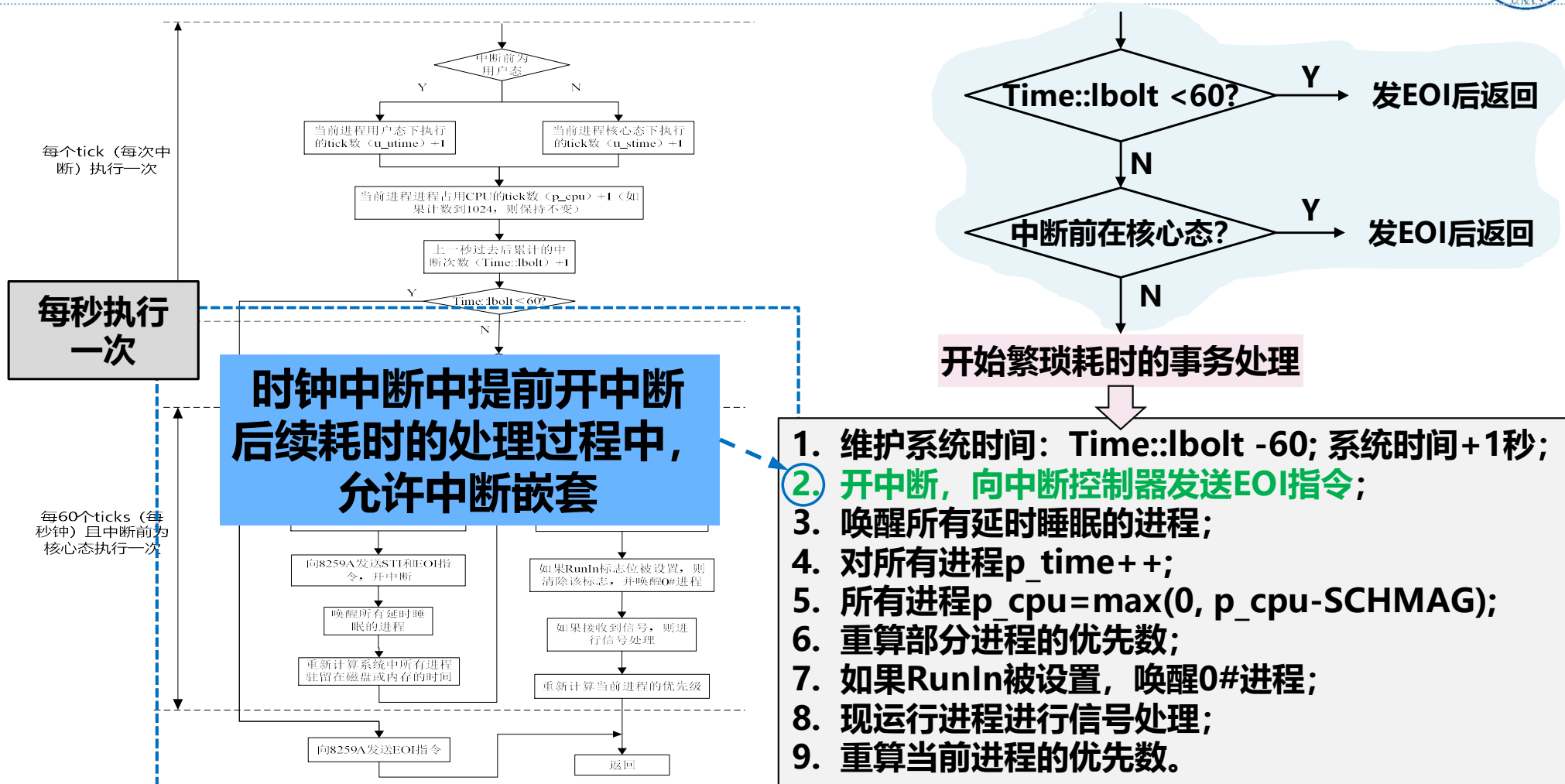
1. 维护系统时间: **Time::lbolt - 60; 系统时间 + 1秒;**
2. 开中断, 向中断控制器发送EOI指令;
3. 唤醒所有延时睡眠的进程;
4. 对所有进程 p_time++ ;
5. 所有进程 $p_cpu = \max(0, p_cpu - SCHMAG)$;
6. 重算部分进程的优先数;
7. 如果RunIn被设置, 唤醒0#进程;
8. 现运行进程进行信号处理;
9. 重算当前进程的优先数.



UNIX时钟中断



时钟中断处理程序的详细流程

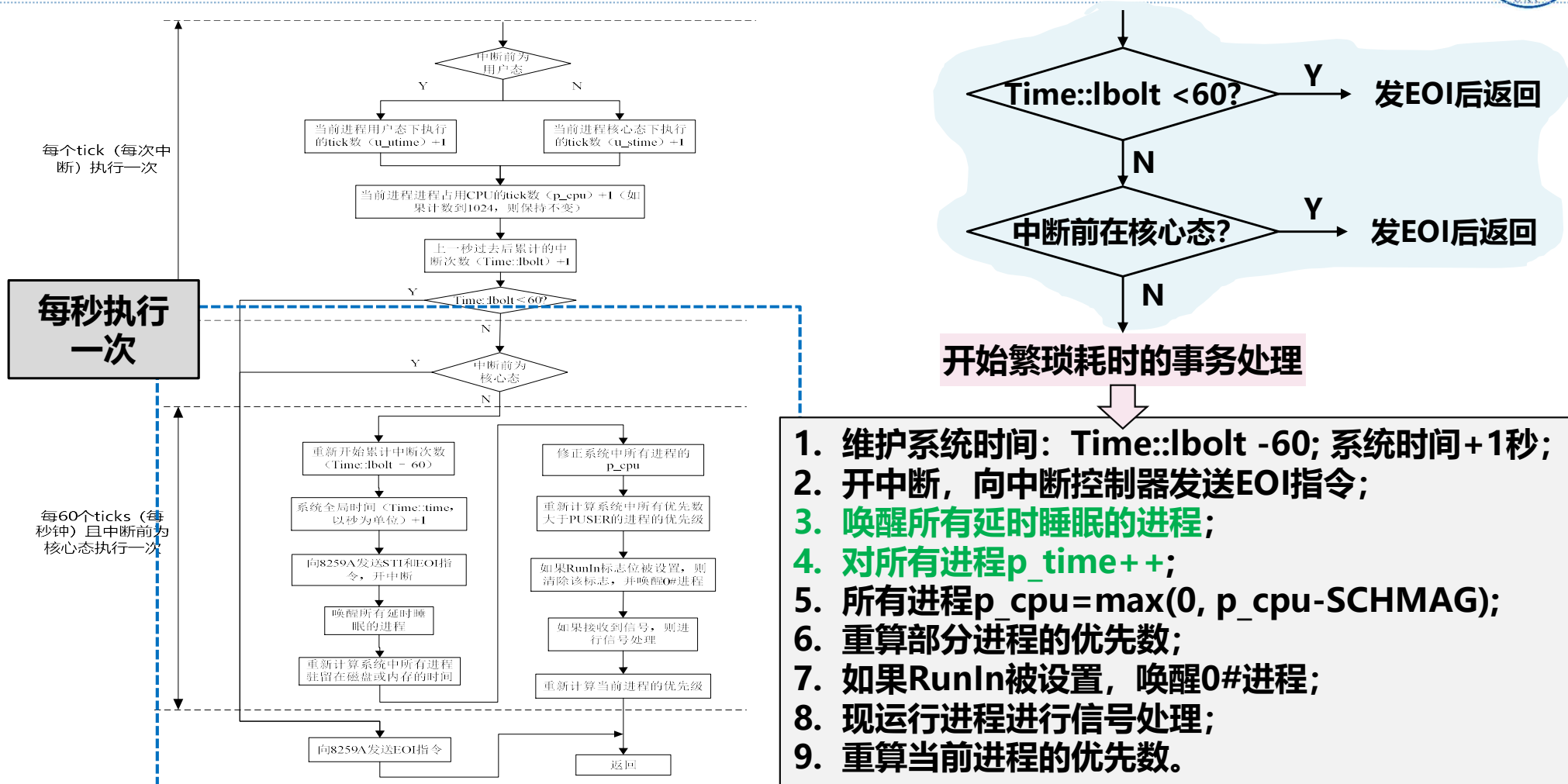




UNIX时钟中断



时钟中断处理程序的详细流程

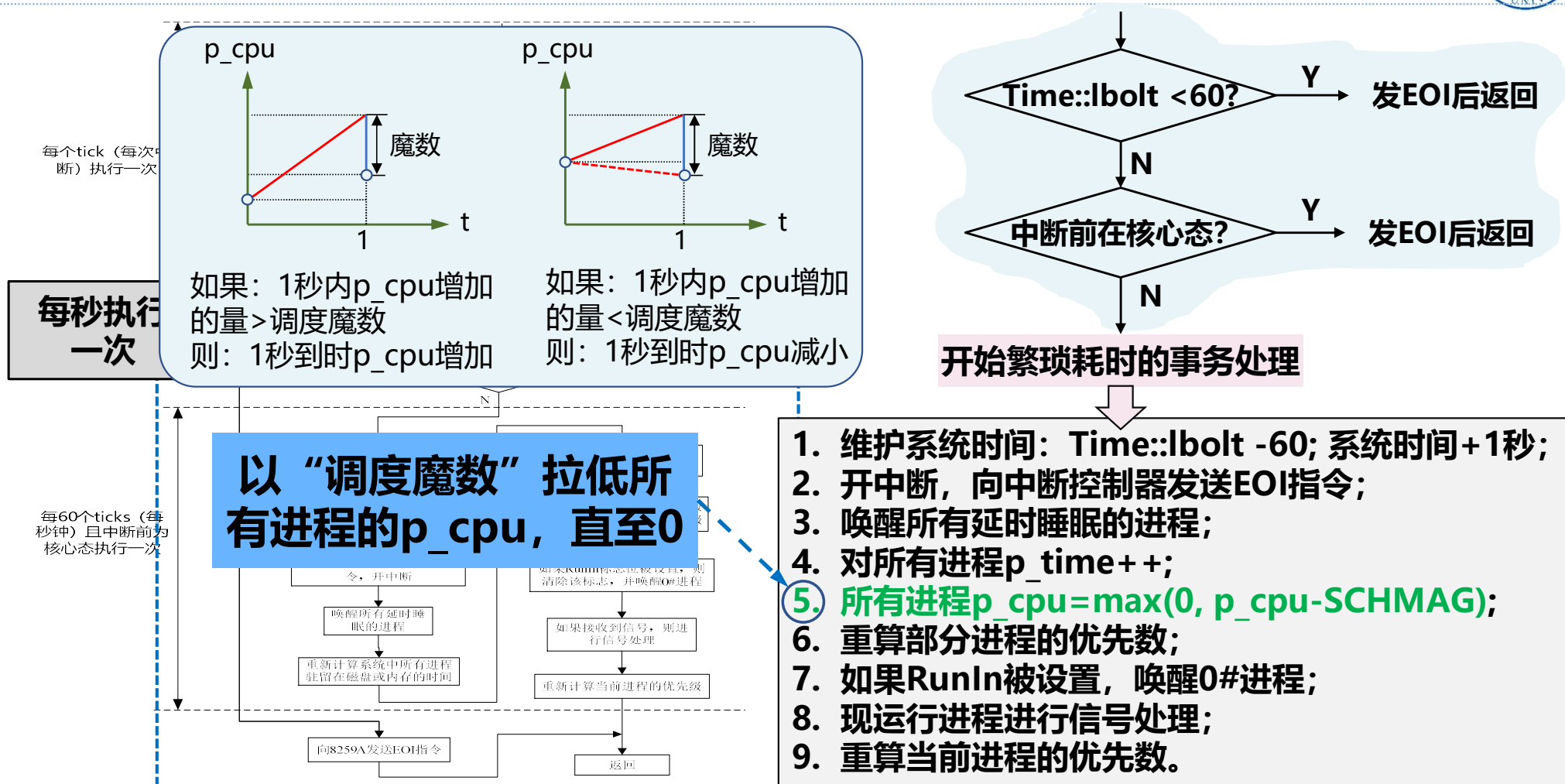




UNIX时钟中断



时钟中断处理程序的详细流程

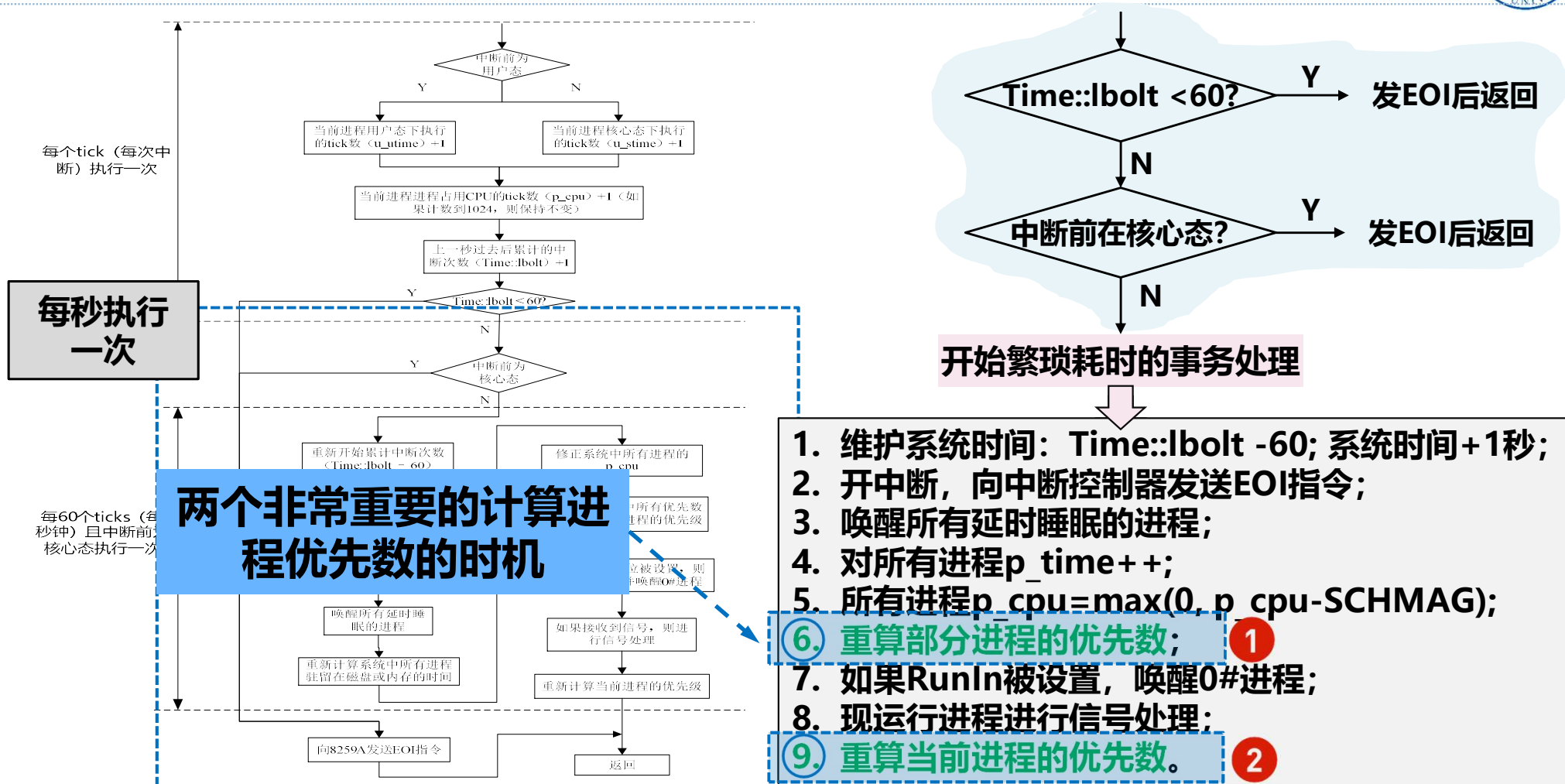




UNIX时钟中断



时钟中断处理程序的详细流程





UNIX时钟中断



进程优先数的计算方法

$$p_pri = \min \{ 255, (p_cpu / 16 + \text{PUSER} + p_nice) \}$$

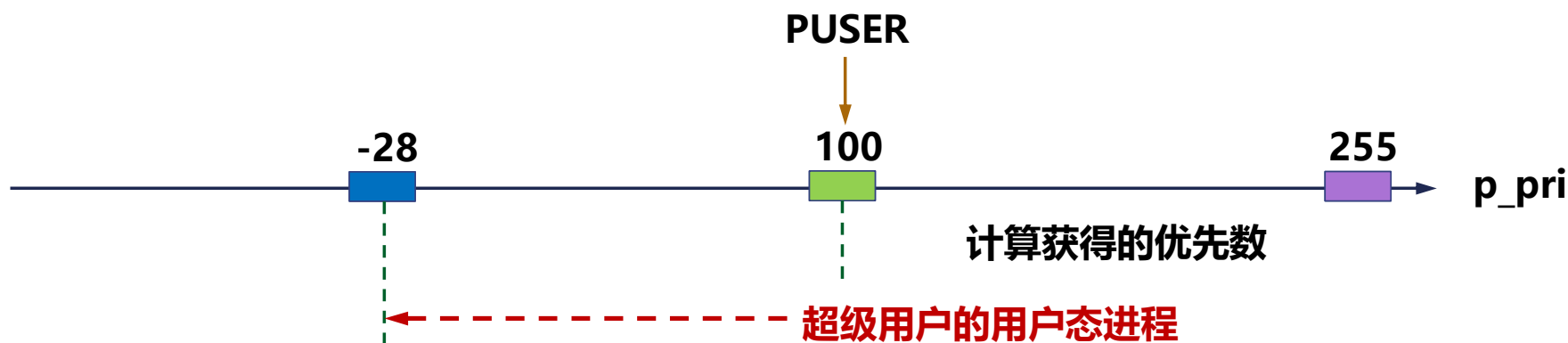
PUSER: 固定偏置常数

`static const int PUSER = 100;`

p_nice: 相对优先程度

值越小, 进程的优先级越高

允许用户通过系统调用设置
超级用户: -128 ~ 20
普通用户: 0 ~ 20





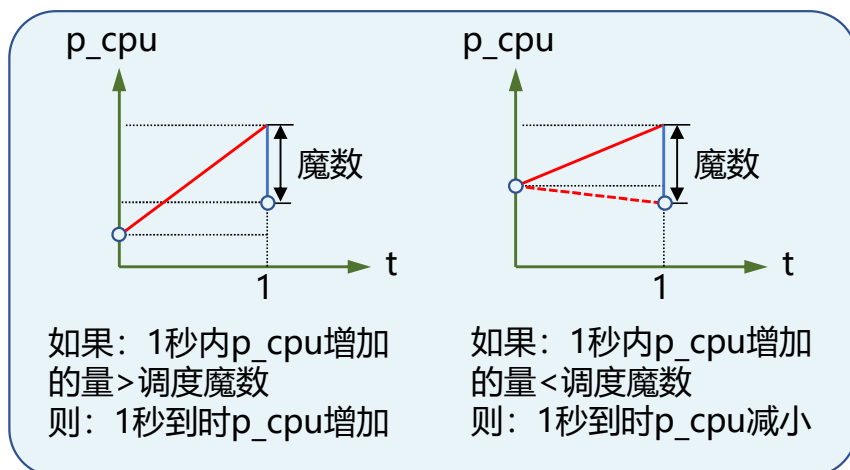
UNIX时钟中断



进程优先数的计算方法

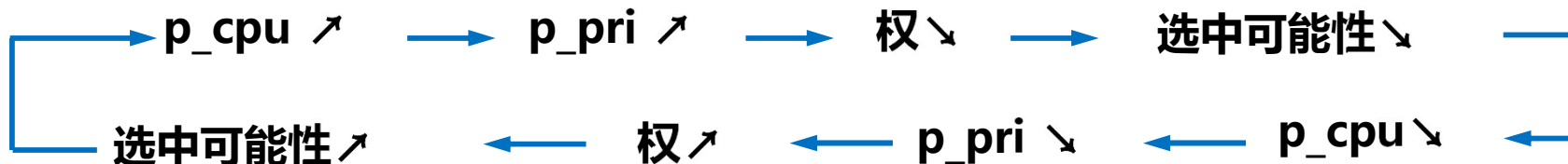
值越小，进程的优先级越高

$$p_pri = \min \{ 255, (p_cpu / 16 + PUSER + p_nice) \}$$



1. 连续占用处理机较长时间的进程：
 $p_cpu \nearrow$ ，优先数 $\nearrow$ ，优先权 $\searrow$
2. 较长时间内未使用处理机的进程：
 $p_cpu \searrow$ ，优先数 $\searrow$ ，优先权 $\nearrow$ ；
3. 虽频繁使用处理机，但每次时间很短的进程：
 $p_cpu \searrow$ ，优先数 $\searrow$ ，优先权 $\nearrow$ 。

防止高者恒高，低者“饥饿”





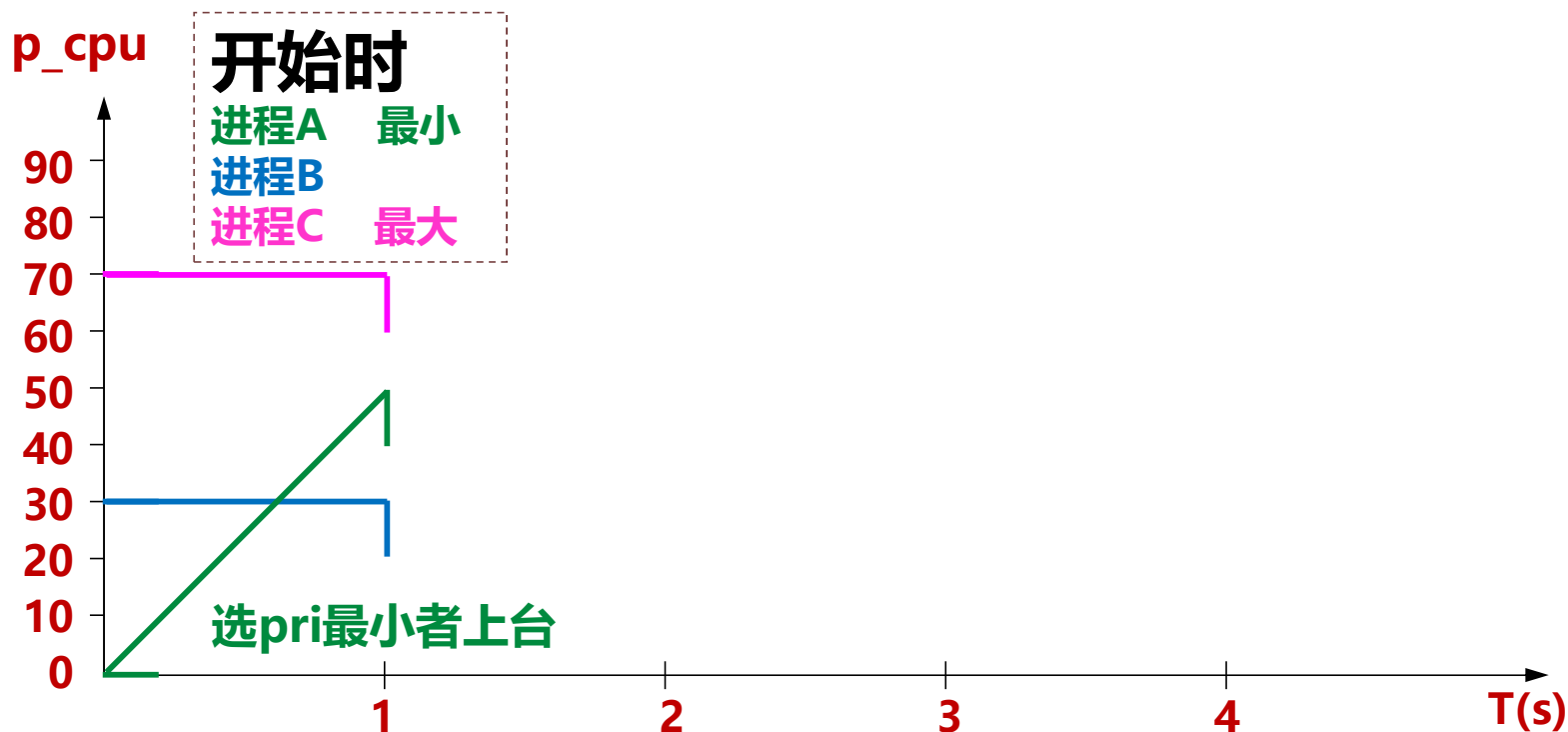
UNIX时钟中断



进程优先数的计算方法

值越小，进程的优先级越高

$$p_pri = \min \{ 255, (p_cpu / 16 + PUSER + p_nice) \}$$





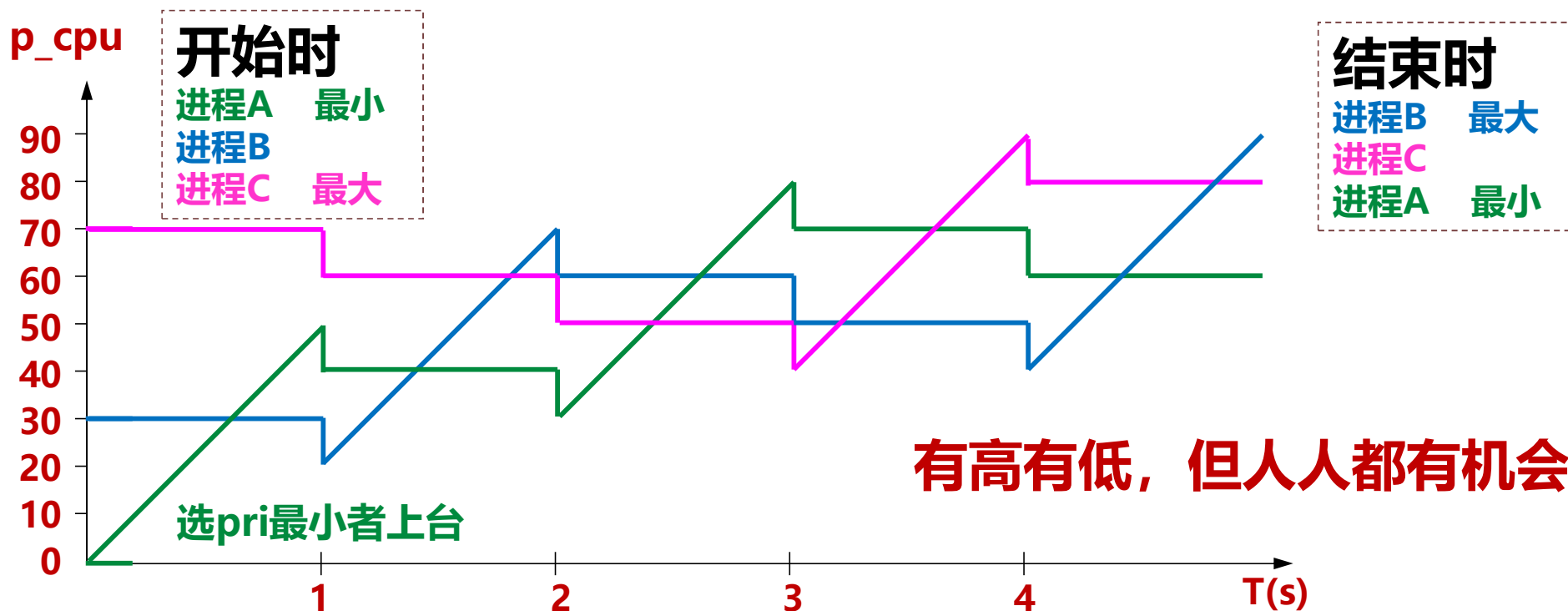
UNIX时钟中断



进程优先数的计算方法

值越小，进程的优先级越高

$$p_pri = \min \{ 255, (p_cpu / 16 + PUSER + p_nice) \}$$

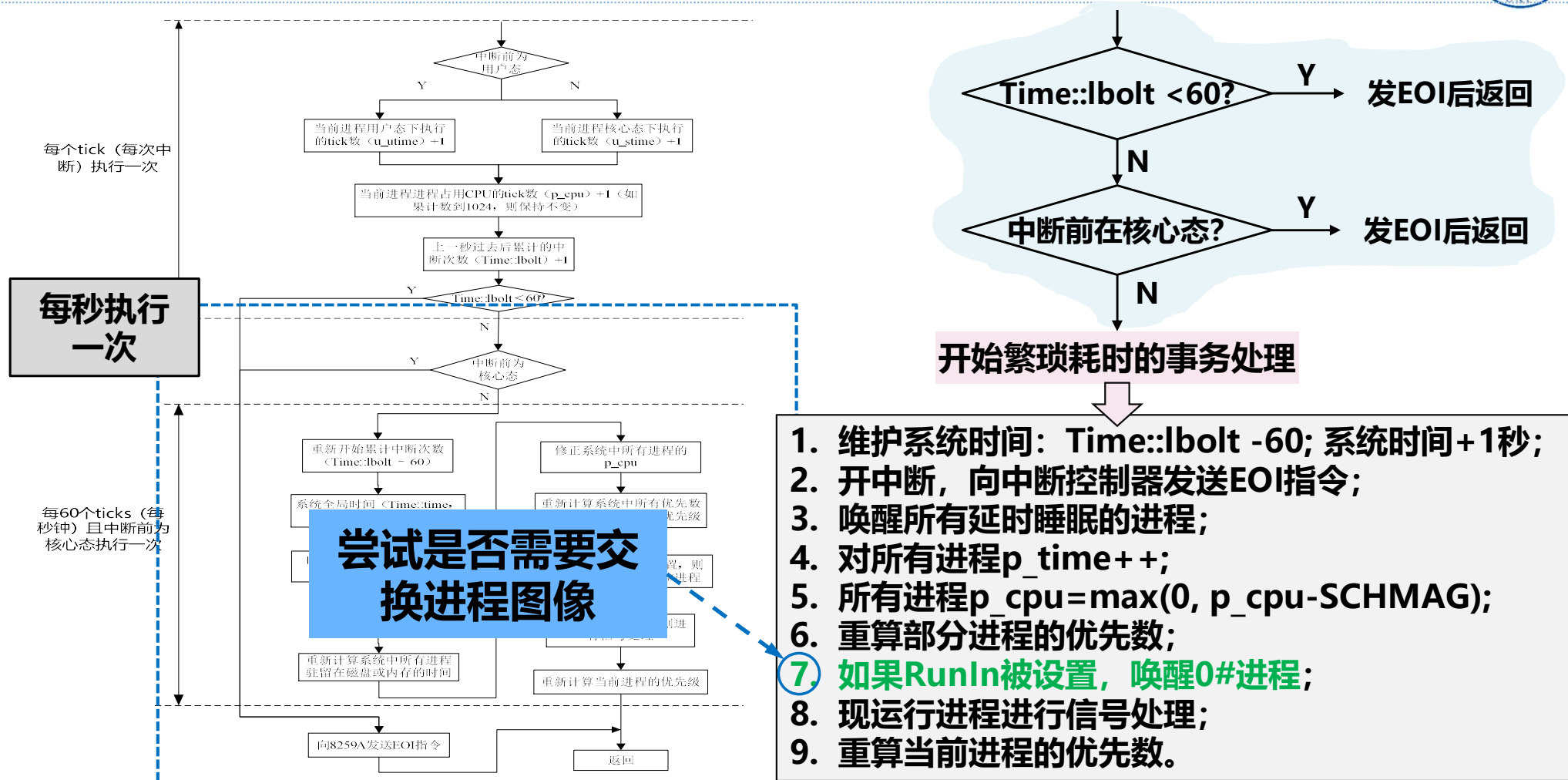




UNIX时钟中断



时钟中断处理程序的详细流程





UNIX时钟中断

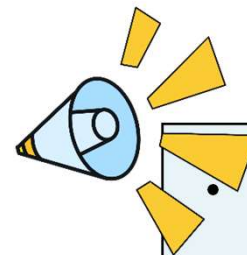


总结一下时钟中断的主要工作

时钟中断以每秒60次的频率定时自动发生



和进程是否发起IO操作无关



- 对p_cpu的处理
- 进程优先数的计算
- 两次计算优先数

每1秒钟一次

每次心跳一次

- u_utime, u_stime的计数
- p_cpu
- lbolt的计数

简单，迅速完成

- 维护系统时间
- 唤醒所有延时睡眠的进程
- 修改所有进程的p_time
- 调整所有进程的p_cpu
- 重算部分进程的优先级
- 可能唤醒0#进程
- 重算当前进程的优先级

繁琐，耗时

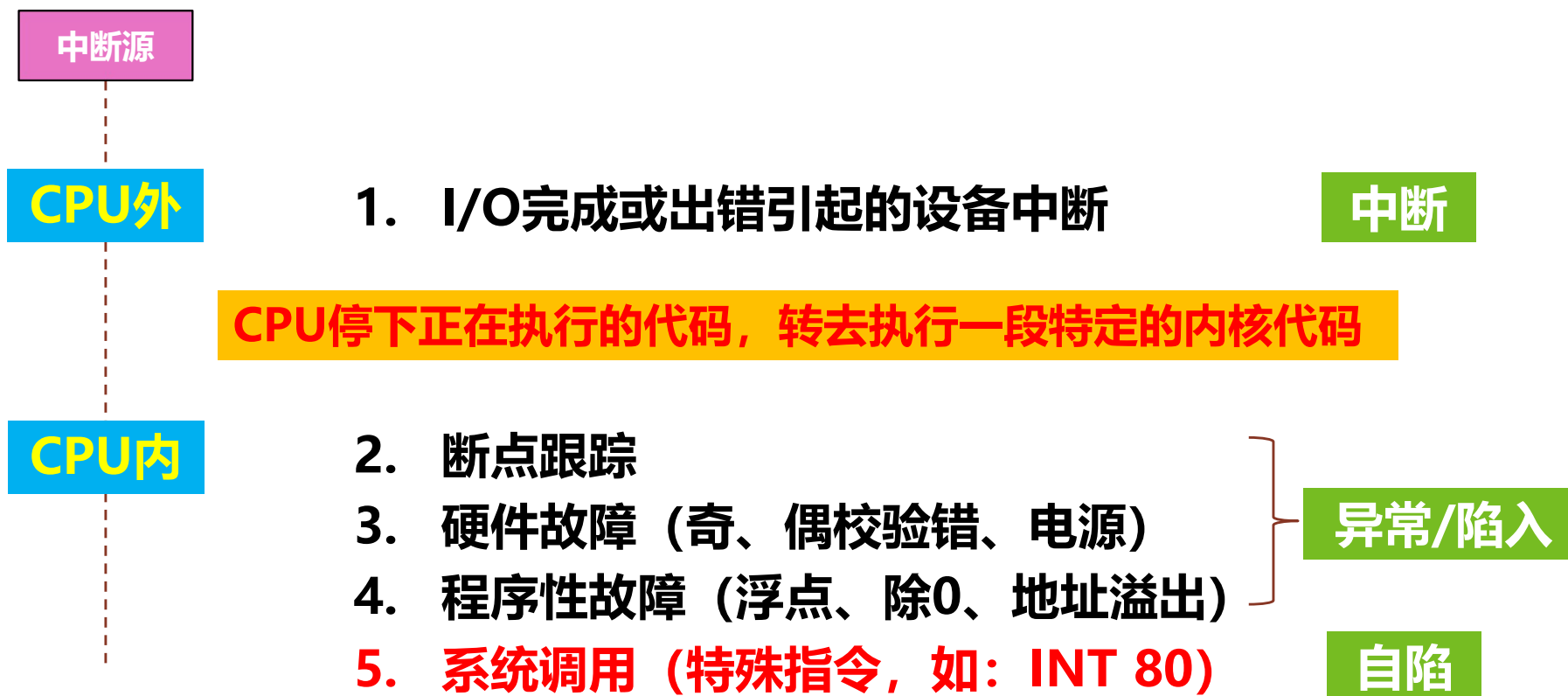
- 先前态是用户态才做
- 提前开中断，EOI

主要内容

- 2.1 进程及其调度控制与死锁
- 2.2 UNIX的进程
- 2.3 中断的基本概念及UNIX中断处理
- 2.4 UNIX时钟中断与系统调用
- 2.5 UNIX的进程调度状态
- 2.6 进程通讯

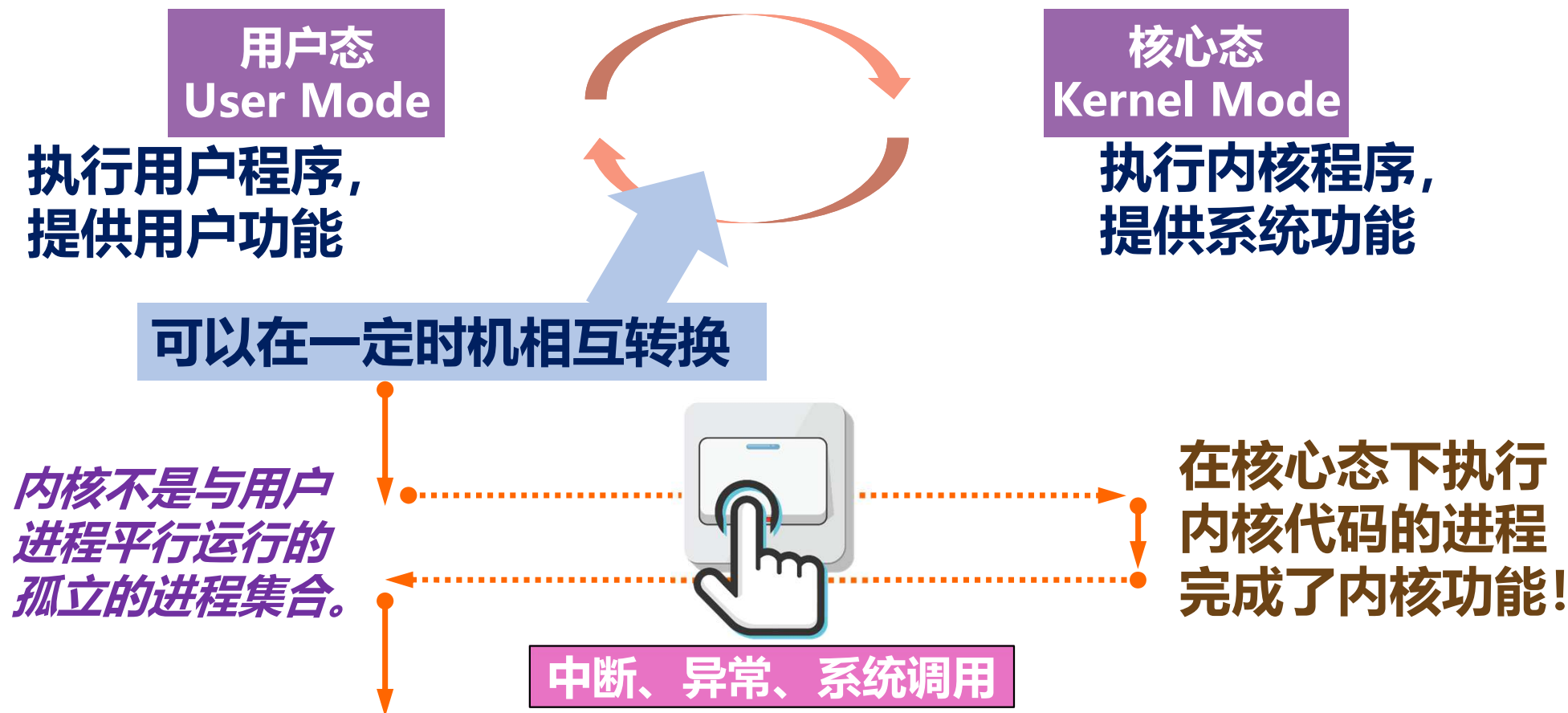


现代计算机系统中，中断的概念被扩展.....





UNIX系统调用





UNIX系统调用



UNIX V6++定义的中断入口程序

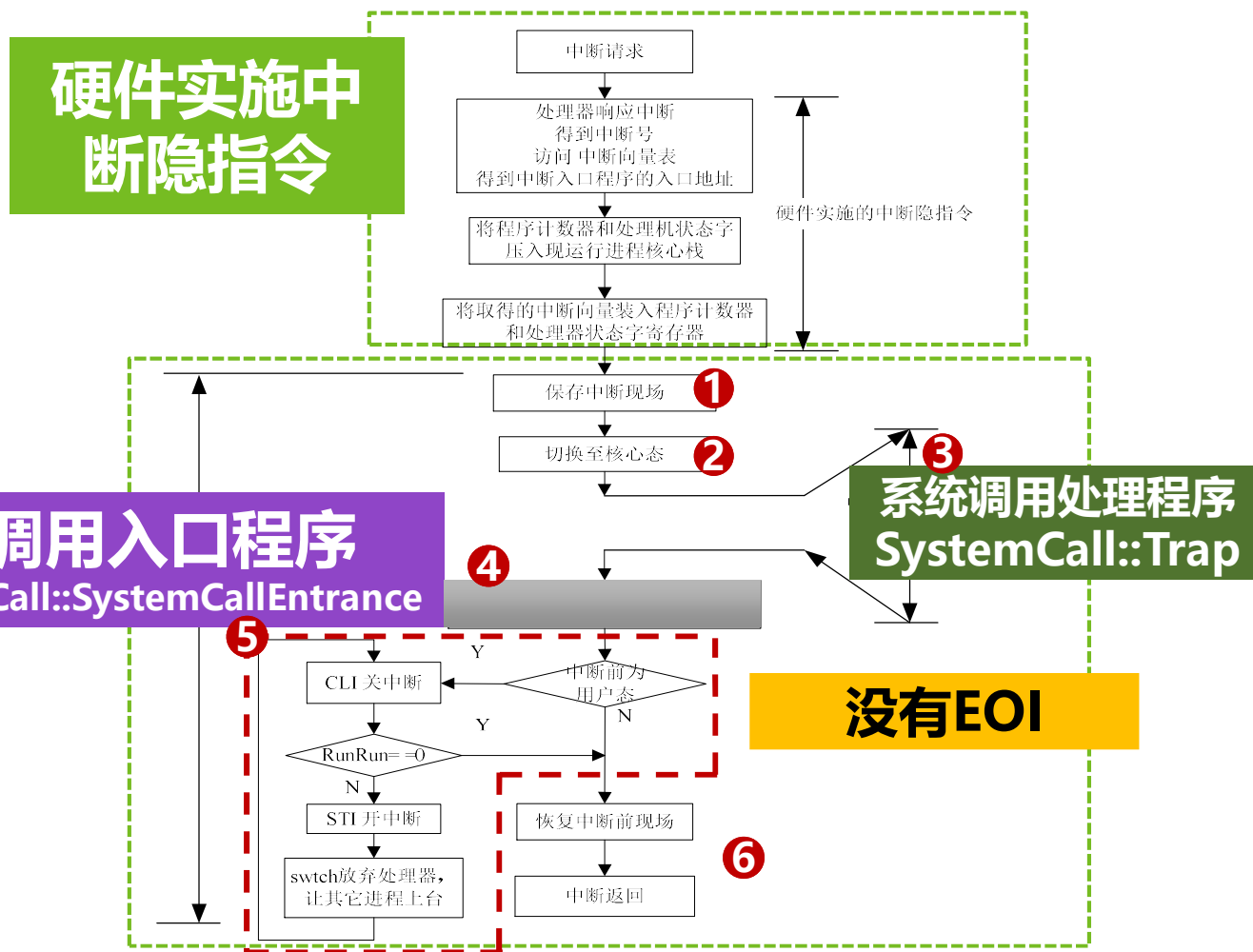
中断号/中断源		中断入口程序	中断处理子程序
0x0	除0错	Exception::DivideErrorEntrance()	Exception::DivideError (struct pt_regs* regs, struct pt_context* context);
0x1	调试异常	Exception::DebugEntrance();	Exception::Debug (struct pt_regs* regs, struct pt_context* context);
0x2	NMI非屏蔽中断	Exception::NMIEntance();	Exception:: NMI (struct pt_regs* regs, struct pt_context* context);
0x3	调试断点	Exception::BreakpointEntrance();	Exception:: Breakpoint (struct pt_regs* regs, struct pt_context* context);
.....			
0x1F	保留异常		
0x20	时钟中断	Time::TimeInterruptEntrance()	void Time::Clock(struct pt_regs* regs, struct pt_context* context)
0x21	键盘中断	KeyboardInterrupt::KeyboardInterruptEntrance()	void Keyboard::KeyboardHandler(struct pt_regs* reg, struct pt_
...	...		
0x2E	硬盘中	...	void ATADriver::ATAHandler(struct pt_regs *reg, struct pt_context *context)
...	...		...
0x80	系统调用	void SystemCall::SystemCallEntrance()	void SystemCall::Trap(struct pt_regs* regs, struct pt_context* context)
...	...	...	...

在中断描述符表的0x80
号中断处登记地址

入口程序跳转至处理程序



UNIX系统调用

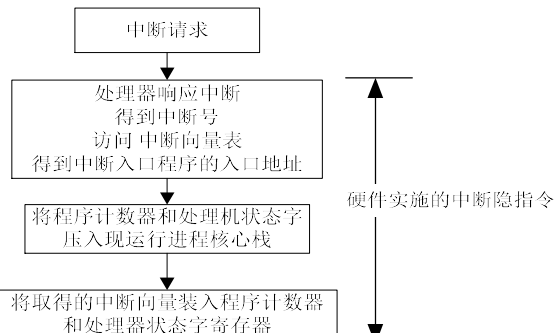




UNIX系统调用



硬件实施中断隐指令



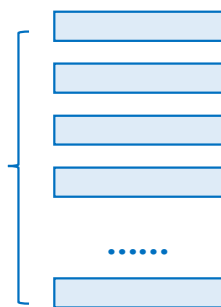
只有一个中断入口
如何完成不同的系
统功能?

系统调用入口程序 SystemCall::SystemCallEntrance

保存中断现场 ①
切换至核心态 ②

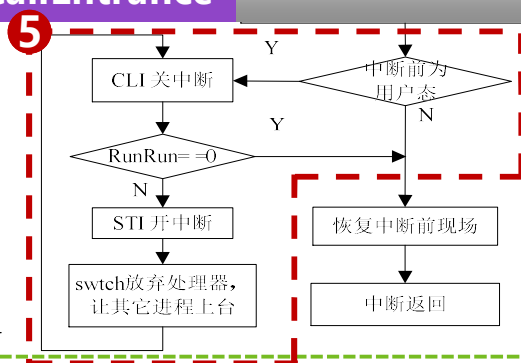
系统调用处理程序 SystemCall::Trap

根据需求
程序散转



一组系统调用处
理子程序

没有EOI





UNIX系统调用



系统调用基本概念

```
struct SystemCallTableEntry
{
    unsigned int count; //系统调用的参数个数
    int(*call)();       //相应系统调用处理函数的指针
};

class SystemCall
{
public:
    static const unsigned int SYSTEM_CALL_NUM = 64;
private:
    static SystemCallTableEntry m_SystemEntranceTable[SYSTEM_CALL_NUM];

public:
    static void SystemCallEntrance();
    static void Trap(struct pt_regs* regs, struct pt_context* context);
    static void Trap1(int (*func)());

private:
    static int Sys_NullSystemCall();
    static int Sys_Rexit();
    static int Sys_Fork();
    .....
};
```

提供一组系统调用处理子程序



UNIX系统调用



```
struct SystemCallTableEntry
{
    unsigned int count; //系统调用的参数个数
    int (*call)();      //相应系统调用处理函数的指针
};
```

```
class SystemCall
{
public:
    static const unsigned int SYSTEM_CALL_NUM = 64;
private:
    static SystemCallTableEntry m_SystemEntranceTable[SYSTEM_CALL_NUM];

public:
    static void SystemCallEntrance();
    static void Trap(struct pt_regs* regs, struct pt_context* context);
    static void Trap1(int (*func)());

private:
    static int Sys_NullSystemCall();
    static int Sys_Rexit();
    static int Sys_Fork();
    .....
};
```

定义一个包含64个分量的系统调用处理子程序入口表

系统调用子程序入口表

0	0	&Sys_NullSystemCall
1	1	&Sys_Rexit
2	0	&Sys_Fork
3	3	&Sys_Read
4	3	&Sys_Write
5	2	&Sys_Open
...		
63	0	&Sys_Nosys

提供一组系统调用处理子程序



UNIX系统调用



```
struct SystemCallTableEntry
{
    unsigned int count; //系统调用的参数个数
    int (*call)();      //相应系统调用处理函数的指针
};
```

```
class SystemCall
{
public:
    static const unsigned int SYSTEM_CALL_NUM = 64;
private:
    static SystemCallTableEntry m_SystemEntranceTable[SYSTEM_CALL_NUM];

public:
    static void SystemCallEntrance();
    static void Trap(struct pt_regs* regs, struct pt_context* context);
    static void Trap1(int (*func)());

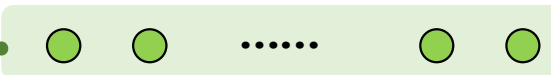
private:
    static int Sys_NullSystemCall();
    static int Sys_Rexit();
    static int Sys_Fork();
    .....
};
```

定义一个包含64个分量的系统调用处理子程序入口表

系统调用子程序入口表		
0	0	&Sys_NullSystemCall
1	1	&Sys_Rexit
2	0	&Sys_Fork
3	3	&Sys_Read
4	3	&Sys_Write
5	2	&Sys_Open
...		
63	0	&Sys_Nosys

参数个数 入口地址

↑ 登记



提供一组系统调用处理子程序



UNIX系统调用



系统调用基本概念

```
struct SystemCallTableEntry
{
    unsigned int count; //系统调用的参数个数
    int (*call)();      //相应系统调用处理函数的指针
};
```

```
class SystemCall
{
public:
    static const unsigned int SYSTEM_CALL_NUM = 64;
private:
    static SystemCallTableEntry m_SystemEntranceTable[SYSTEM_CALL_NUM];

public:
    static void SystemCallEntrance();
    static void Trap(struct pt_regs* regs, struct pt_context* context);
    static void Trap1(int (*func)());

private:
    static int Sys_NullSystemCall();
    static int Sys_Rexit();
    static int Sys_Fork();
    .....
};
```

定义一个包含64个分量的系统调用处理子程序入口表

系统调用号	参数个数	入口地址
0	0	&Sys_NullSystemCall
1	1	&Sys_Rexit
2	0	&Sys_Fork
3	3	&Sys_Read
4	3	&Sys_Write
5	2	&Sys_Open
...		
63	0	&Sys_Nosys

系统调用号

参数个数

入口地址

↑ 登记

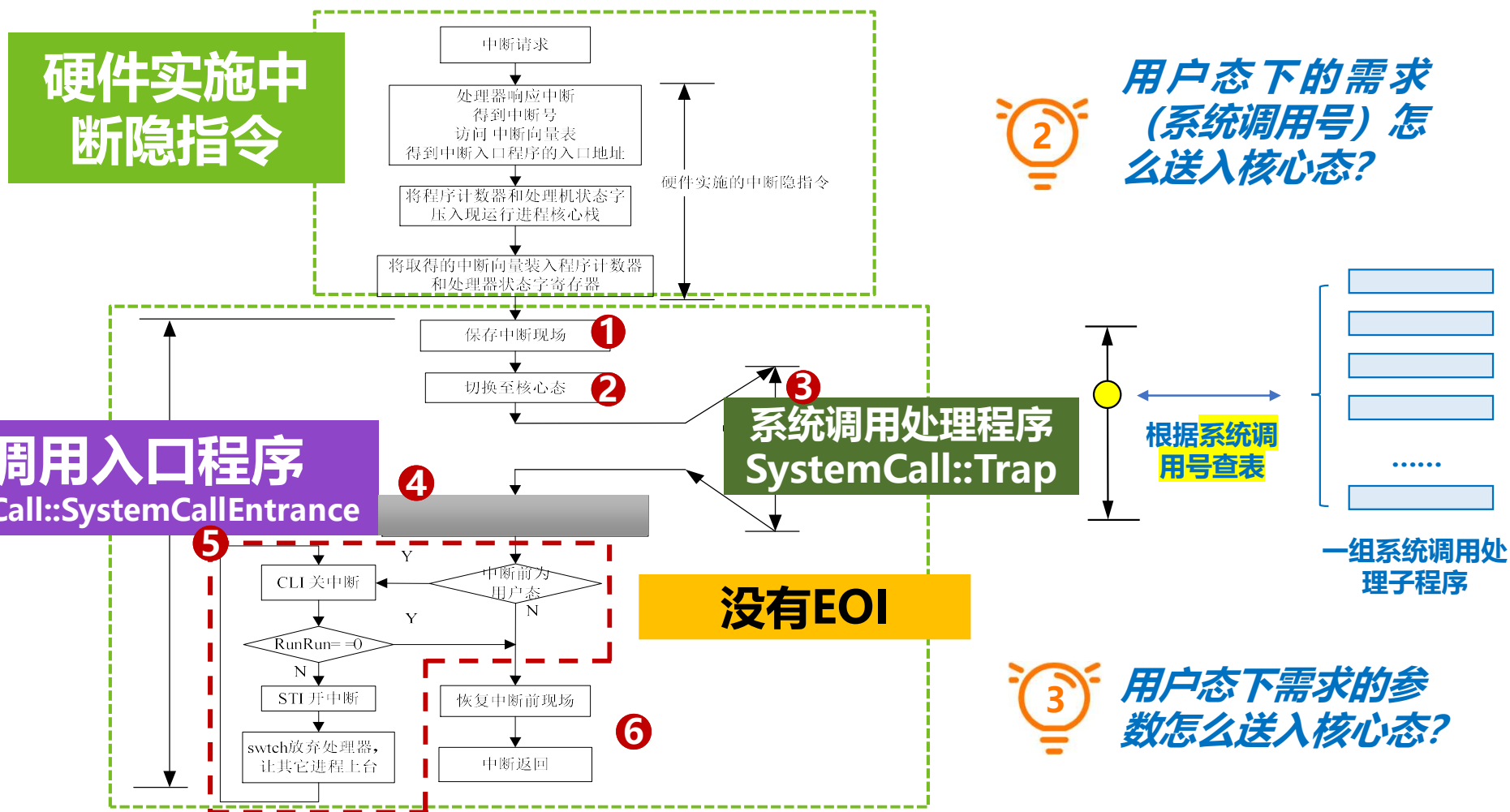
提供一组系统调用处理子程序



UNIX系统调用



系统调用基本概念





UNIX系统调用



用户态 User Mode

```
#include <fcntl.h>
char buffer[2048];
int version = 1;
main( argc, argv)
int  argc;
char *argv[];
{
    int fdold, fdnew;

    .....;
    copy(fdold, fdnew);
    exit(0);
}
copy( old, new)
int old, new;
{
    int count;
    while ( (count = read(old, buffer, sizeof(buffer)))>0)
        write ( new, buffer, count);
}
```



UNIX系统调用



用户态 User Mode

```
#include <fcntl.h>
char buffer[2048];
int version = 1;
main( argc, argv)
int  argc;
char *argv[];
{
    int fdold, fdnew;

    .....;
    copy(fdold, fdnew);
    exit(0);
}
copy( old, new)
int old, new;
{
    int count;
    while ( (count = read(old, buffer, sizeof(buffer)))>0)
        write ( new, buffer, count);
}
```



UNIX系统调用



用户态 User Mode

```
#include <fcntl.h>
char buffer[2048];
int version = 1;
```

```
main( argc, argv)
```

```
int  argc;
```

```
char *argv[];
```

```
{
```

```
    int fdold, fdnew;
```

```
    .....
```

```
    copy(fdold, fdnew);
```

```
    exit(0);
```

```
}
```

```
copy( old, new)
```

```
int old, new;
```

```
{
```

```
    int count;
```

```
    while ( (count = read(old, buffer, sizeof(buffer)))>0)
```

```
        write ( new, buffer, count);
```

```
}
```

C语言的标准库函数

read (fd, buf, nbytes)

INT 0x80

0x80号中断



UNIX系统调用



```
#include <fcntl.h>
char buffer[2048];
int version = 1;
main( argc, argv)
int  argc;
char *argv[];
{
    int fdold, fdnew;

    .....;
    copy(fdold, fdnew);
    exit(0);
}
copy( old, new)
int old, new;
{
    int count;
    while ( (count = read(old, buffer, sizeof(buffer)))>0)
        write ( new, buffer, count);
}
```

用户态
User Mode

C语言的标准库函数

read (fd, buf, nbytes)

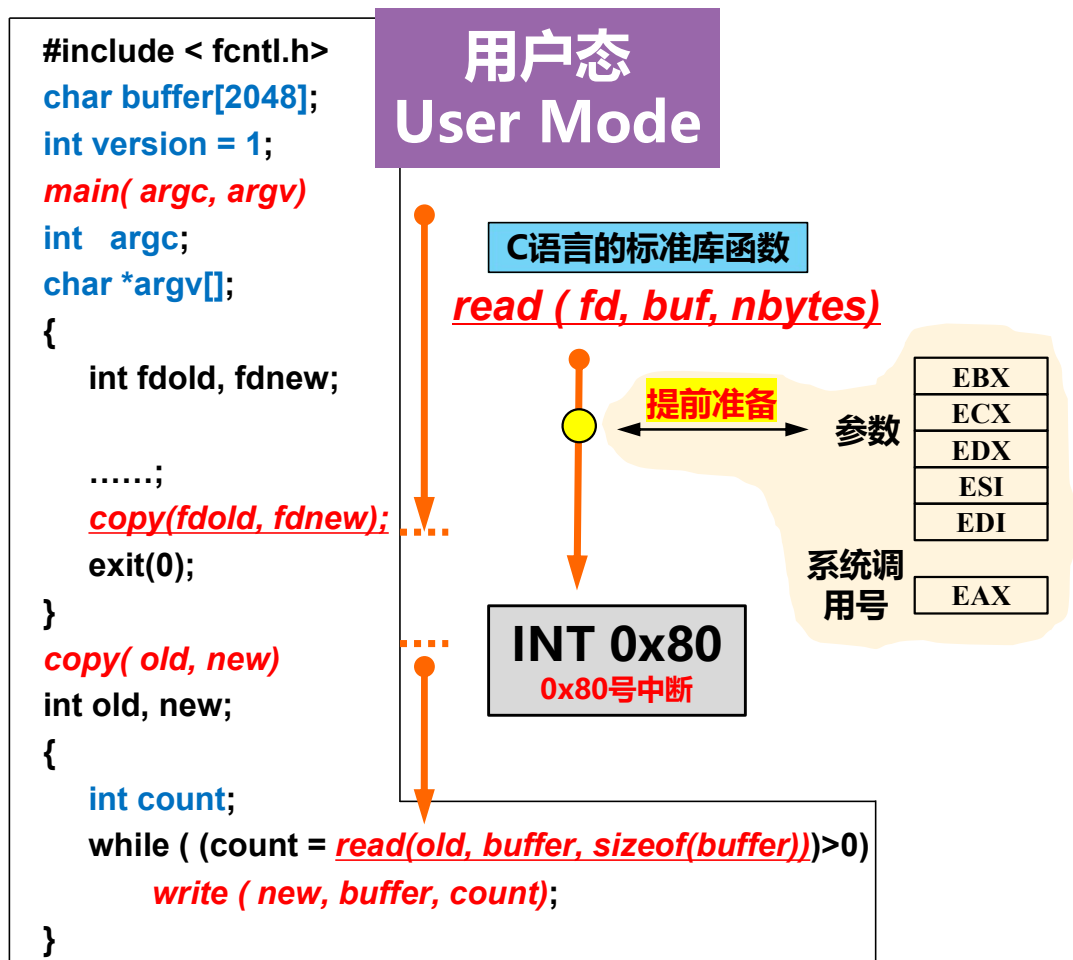
INT 0x80
0x80号中断

核心态
Kernel Mode

系统调用处理



UNIX系统调用



核心态 Kernel Mode

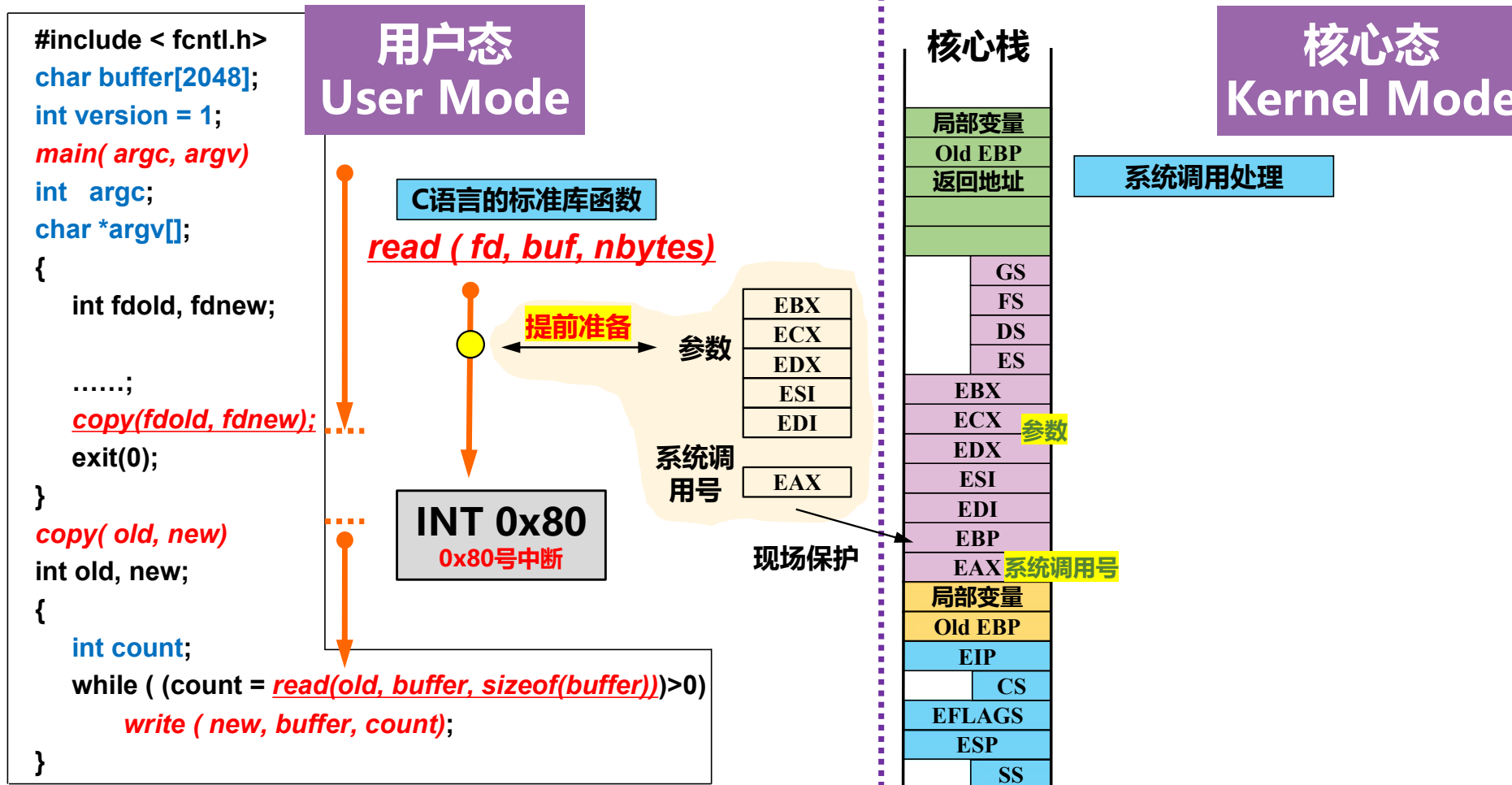
系统调用处理



UNIX系统调用



系统调用基本概念

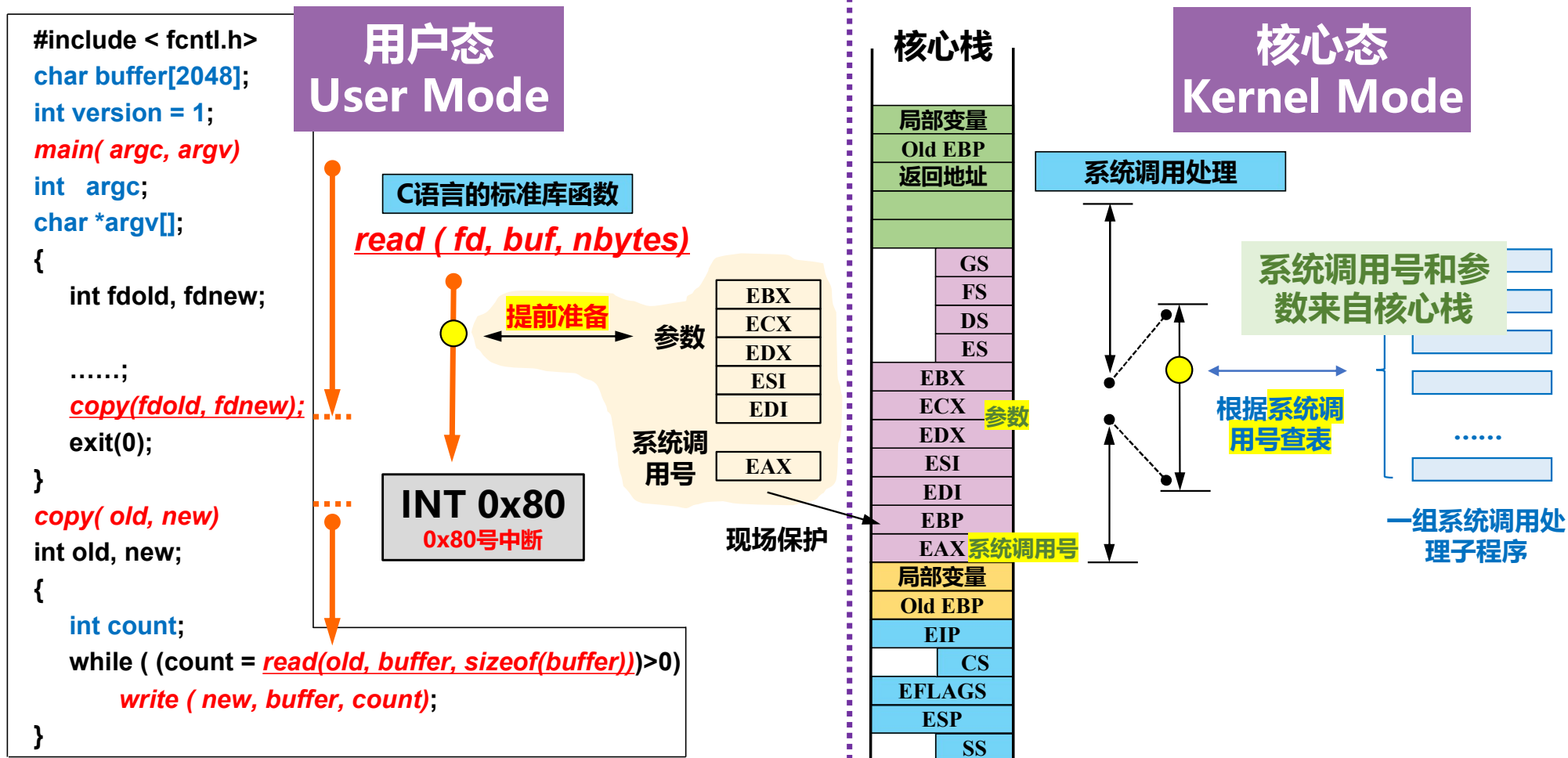




UNIX系统调用



系统调用基本概念

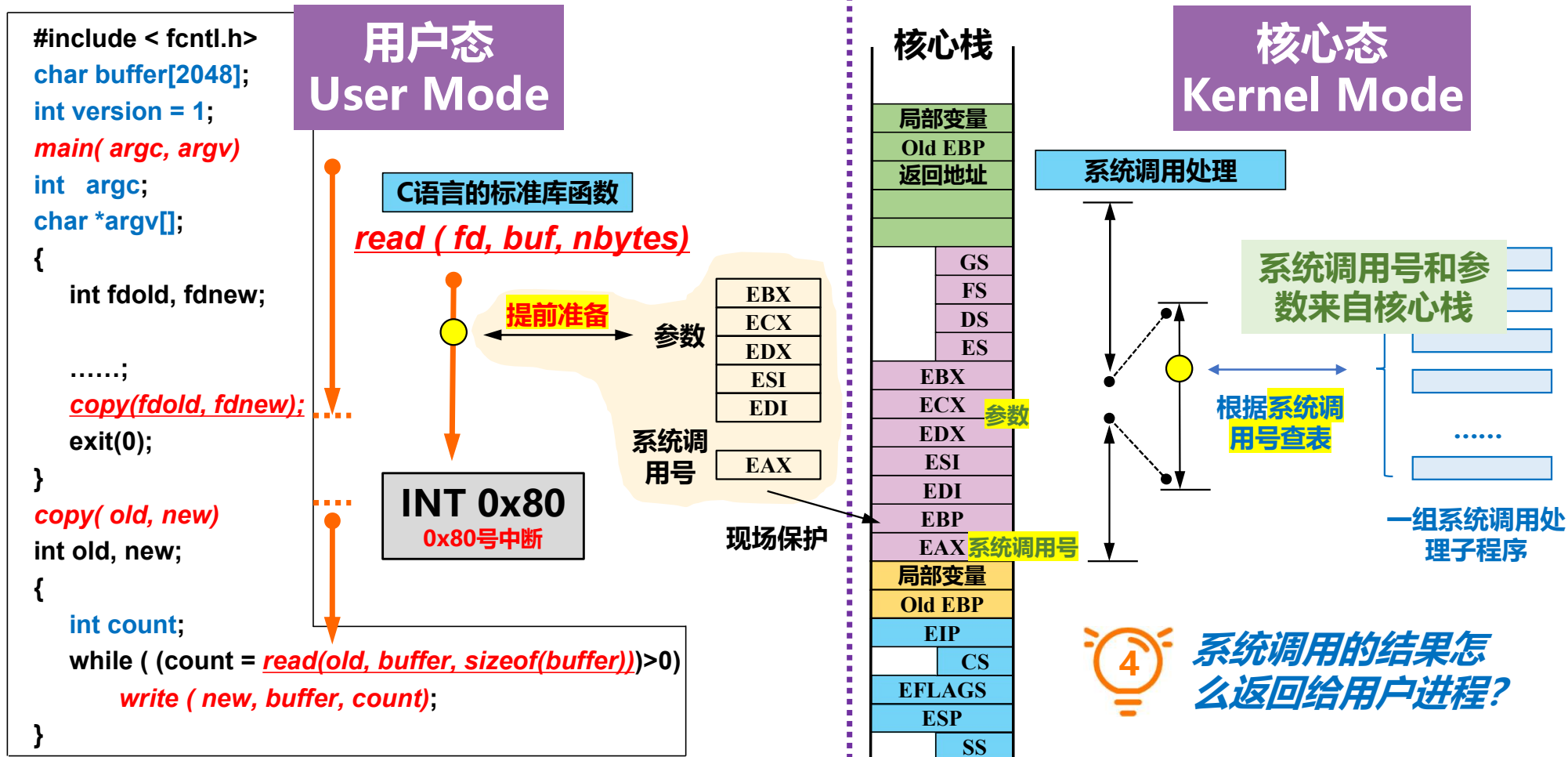




UNIX系统调用

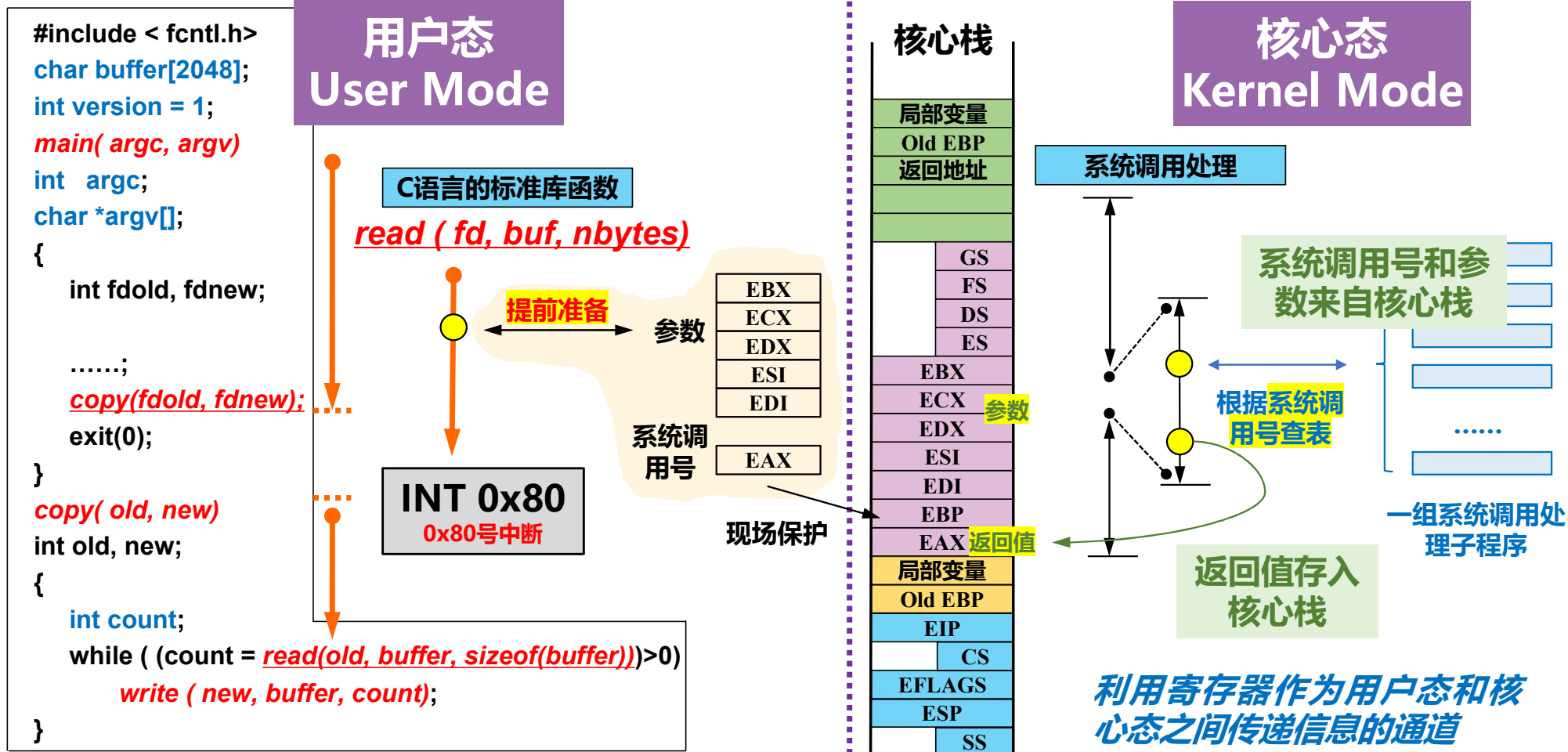


系统调用基本概念





UNIX系统调用





UNIX系统调用



以打开文件为例

```
fd = open( "fileName" , mode);  
if ( fd < 0 ) {  
    printf( "Can not open file!" );  
    exit();  
}
```

每个系统调用均唯一对应于一个高级语言用户库函数（钩子函数）。运行在用户态。

```
int open(char* pathname, unsigned int mode)
```

```
{  
    int res;  
    __asm__ __volatile__ ("int $0x80" : "=a"(res) : "a"(5), "b"(pathname), "c"(mode));  
    if ( res >= 0 )  
        return res;  
    return -1;  
}
```

1. 系统调用号存入EAX

3. 执行INT 0x80指令，引发80号中断

4. 将EAX寄存器带回的返回值赋给res

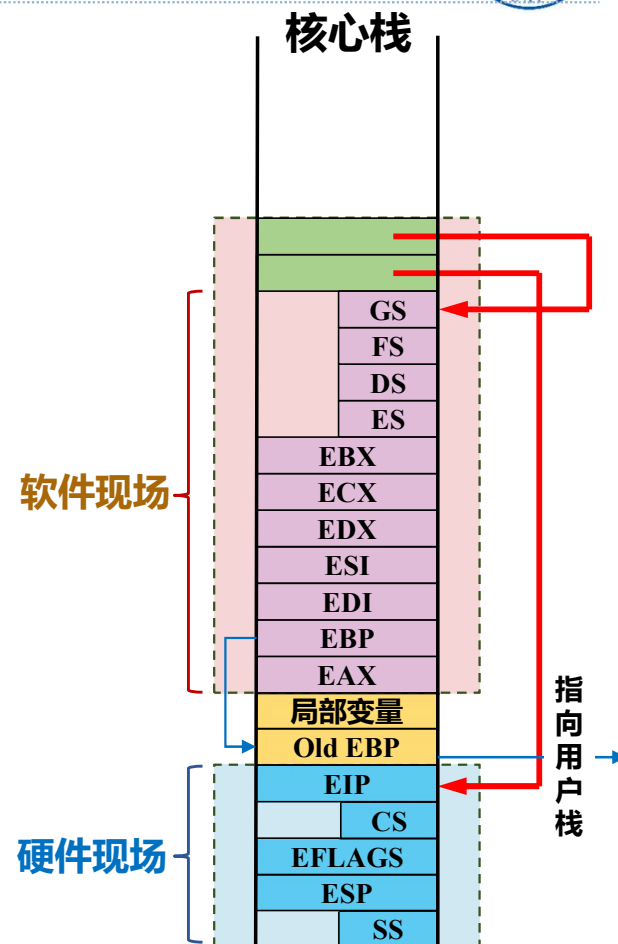
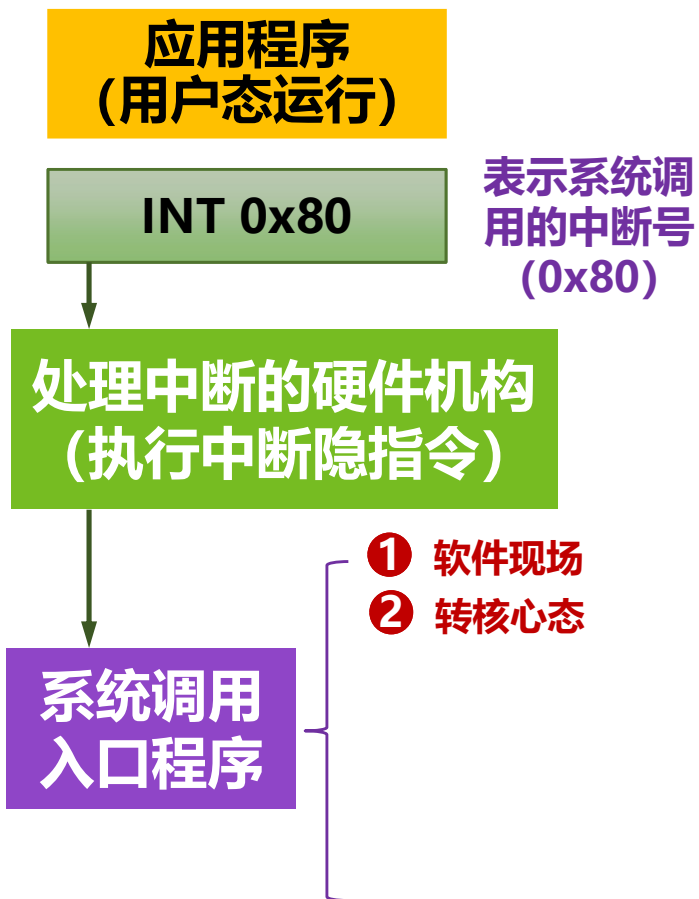
2. 系统调用参数依次写入寄存器EBX, ECX, EDX, ESI和EDI（最多5个）



UNIX系统调用



系统调用详细流程



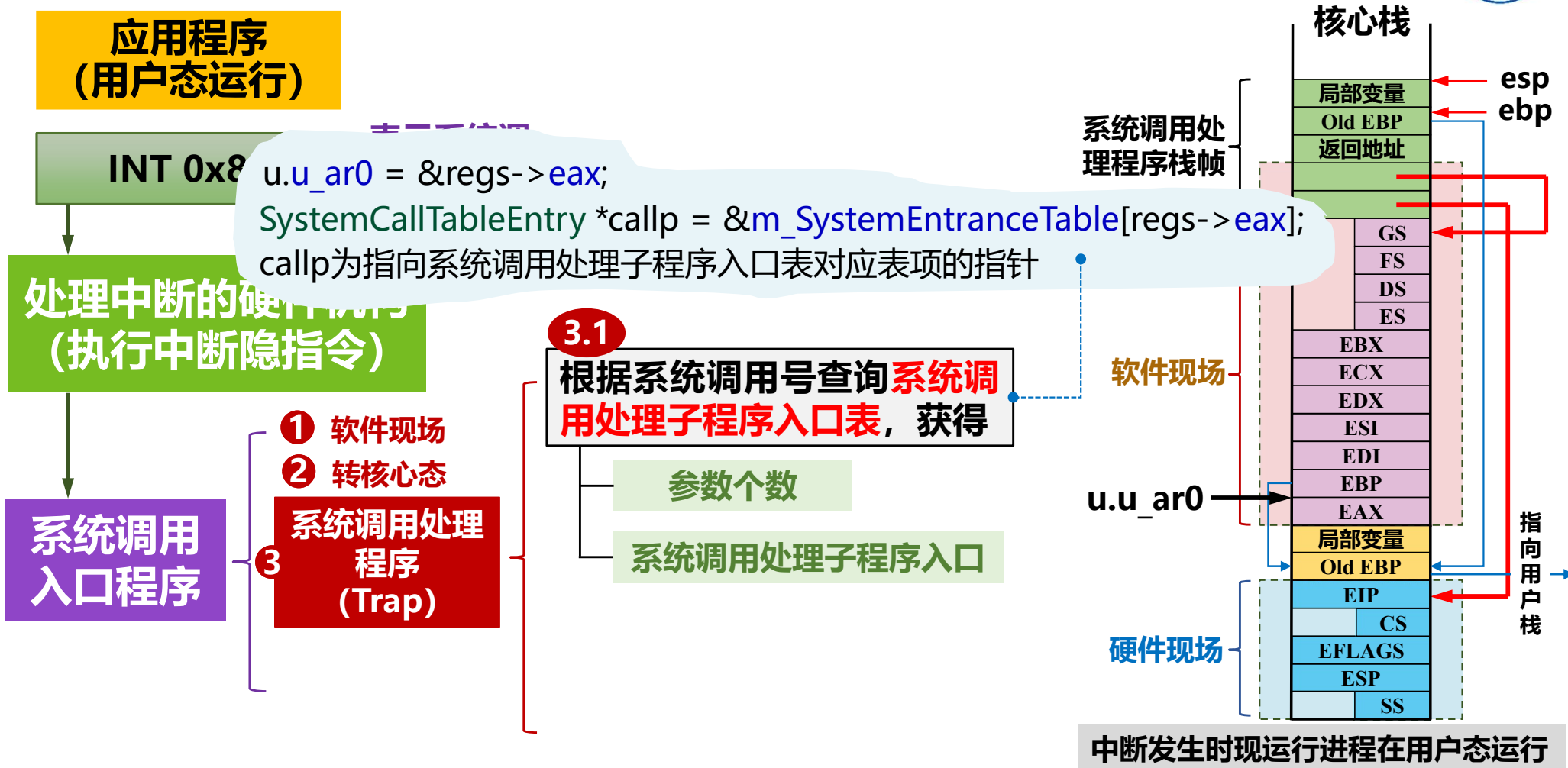
中断发生时现运行进程在用户态运行



UNIX系统调用



系统调用详细流程



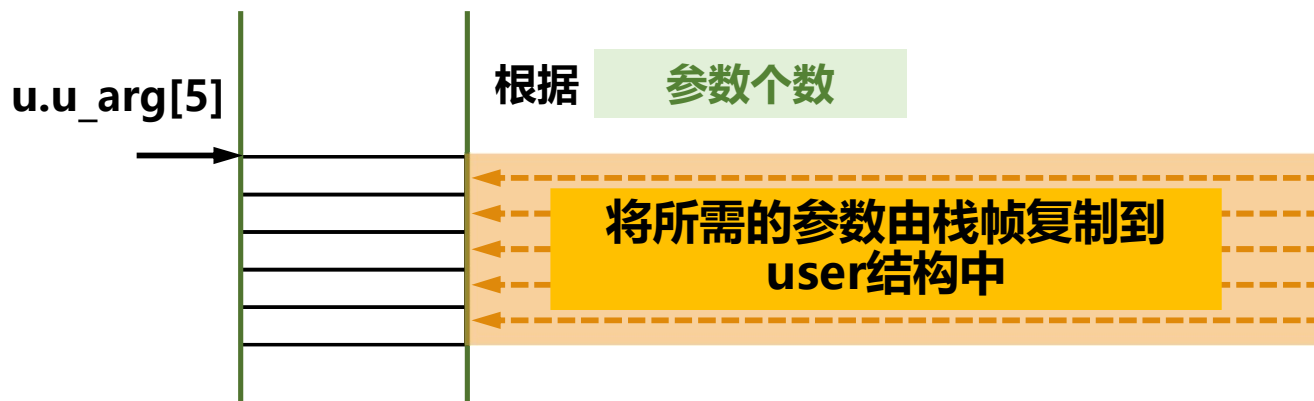


UNIX系统调用

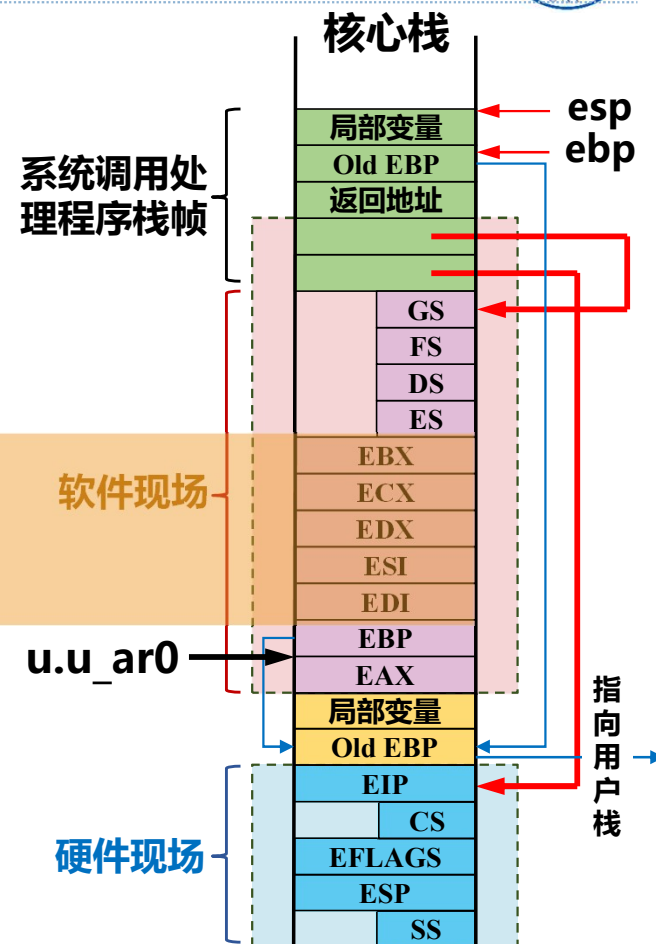


系统调用详细流程

进程的user结构



```
unsigned int * syscall_arg = (unsigned int *)&regs->ebx;
for( unsigned int i = 0; i < callp->count; i++ )
{
    u.u_arg[i] = (int)(*syscall_arg++);
}
```



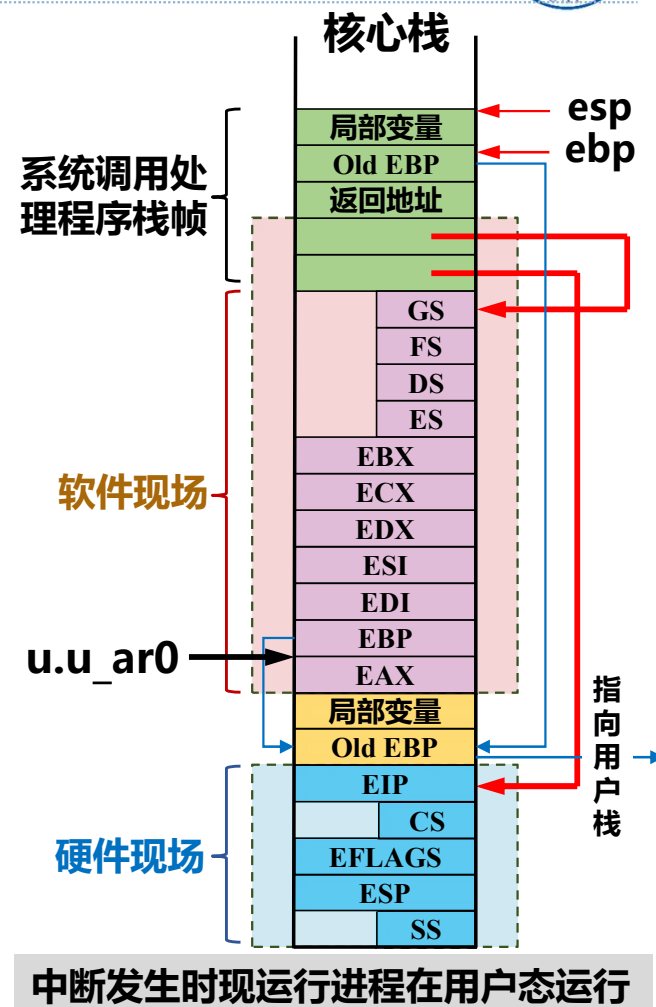
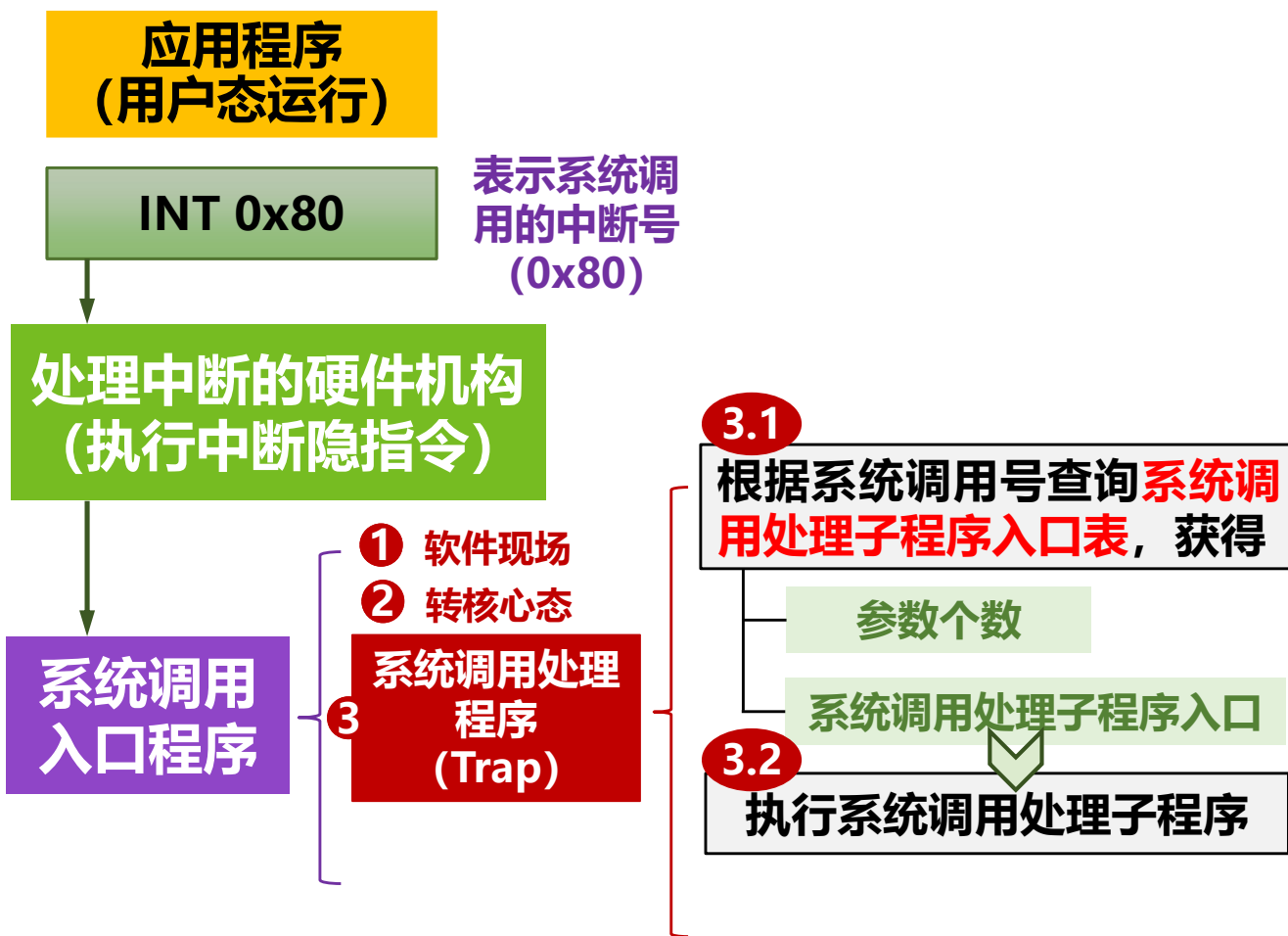
中断发生时现运行进程在用户态运行



UNIX系统调用



系统调用详细流程

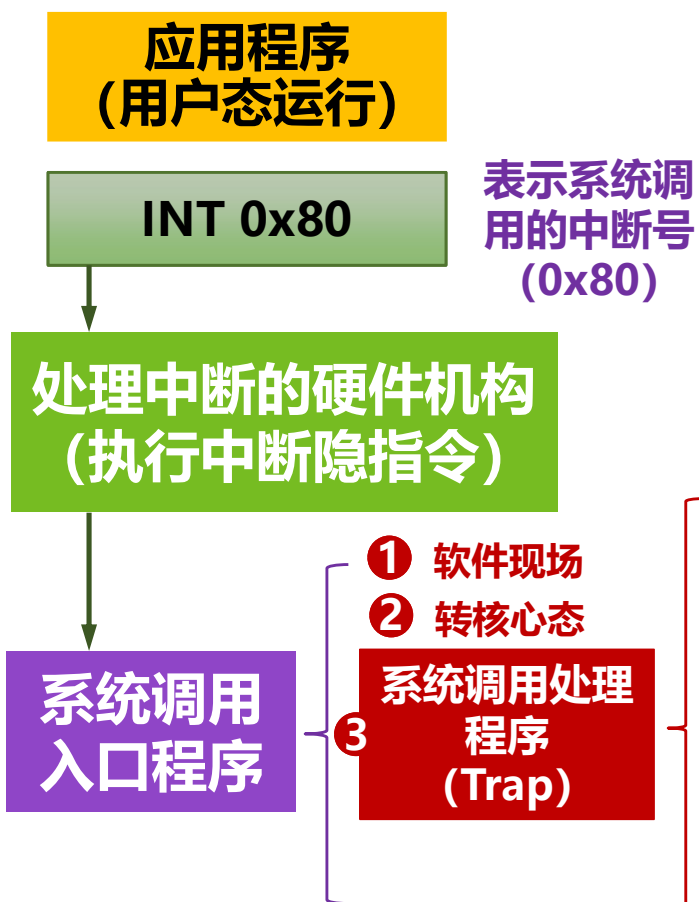




UNIX系统调用



系统调用详细流程



功能虽不同，但遵循以下规则：

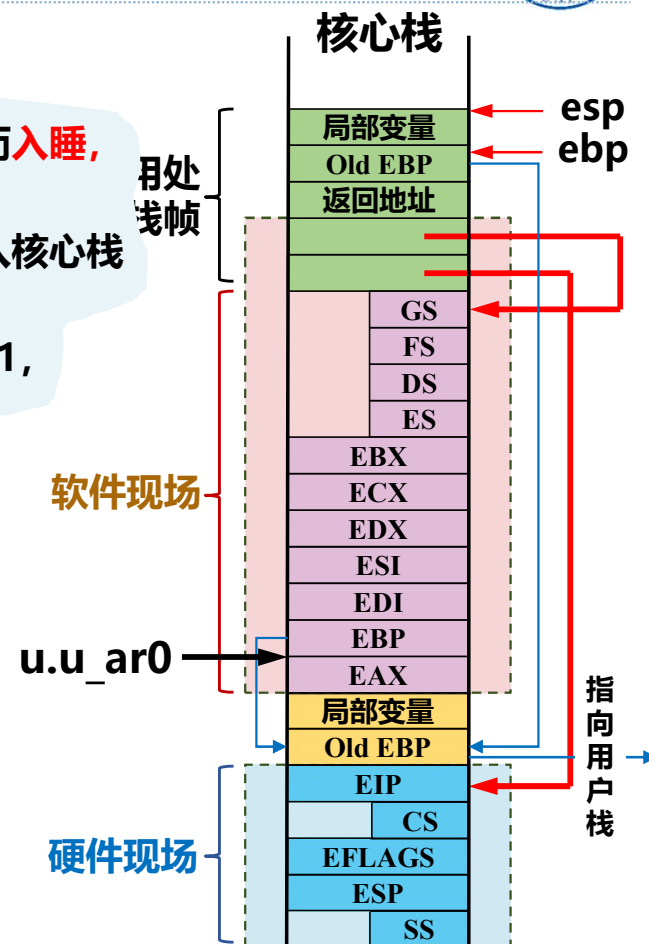
1. 参数取自user结构；
2. 进程可能会因为使用系统资源而**入睡**，**下台**，等待被唤醒后重新上台；
3. 如果系统调用成功，返回值填入核心栈中保存EAX的位置；如果系统调用失败，该位置 = 1，置系统调用出错码 u_error

3.1 根据系统调用号查询系统调用处理子程序入口表，获得

参数个数

系统调用处理子程序入口

3.2 执行系统调用处理子程序



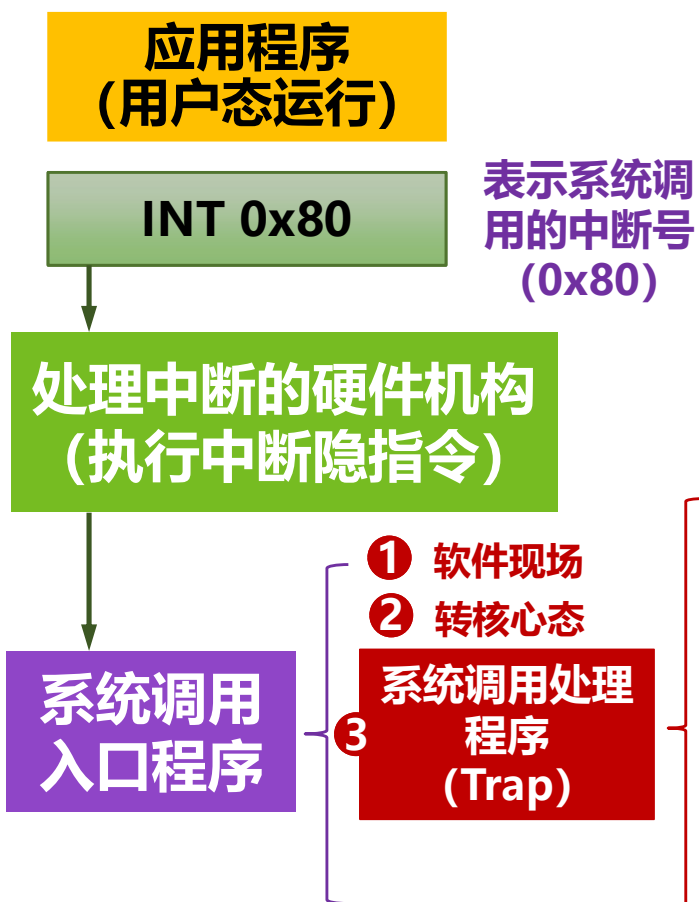
中断发生时现运行进程在用户态运行



UNIX系统调用

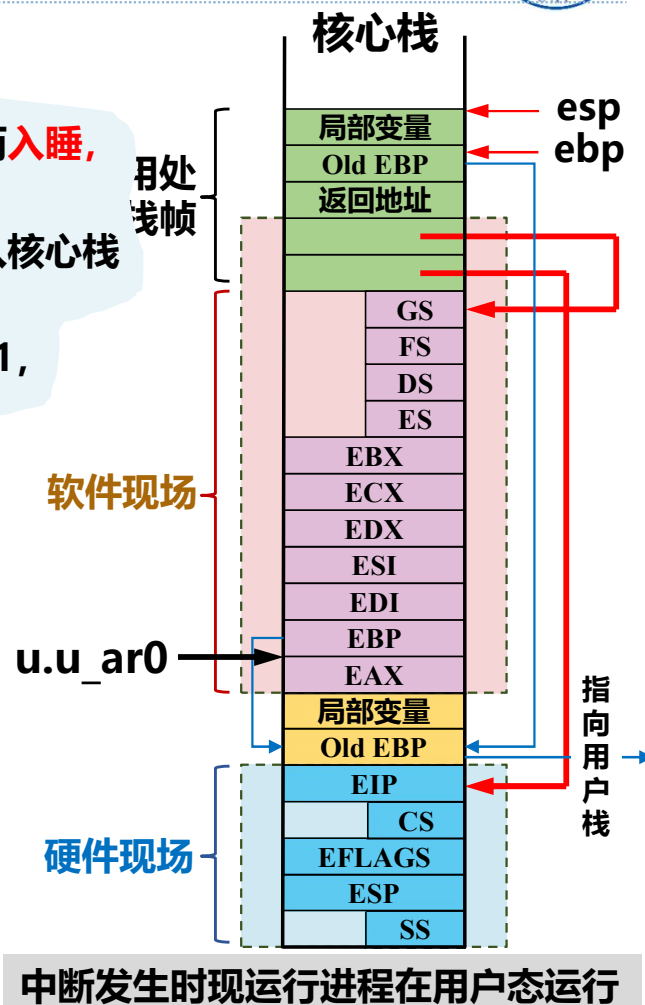
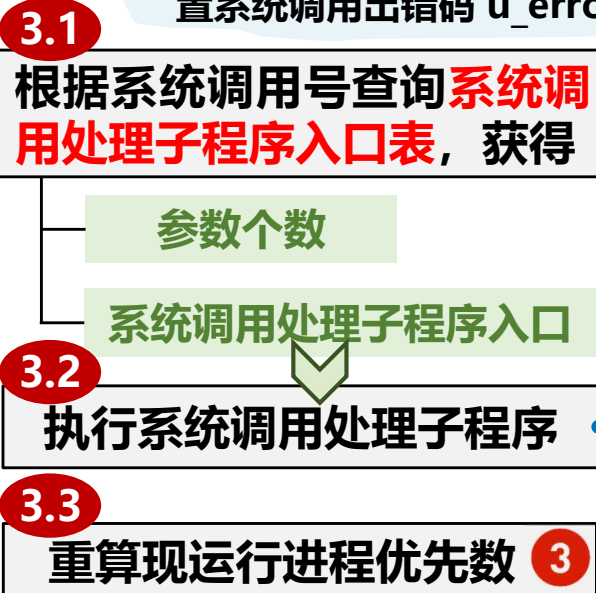


系统调用详细流程



功能虽不同，但遵循以下规则：

1. 参数取自user结构；
2. 进程可能会因为使用系统资源而**入睡**，**下台**，等待被唤醒后重新上台；
3. 如果系统调用成功，返回值填入核心栈中保存EAX的位置；如果系统调用失败，该位置 = 1，置系统调用出错码 u_error

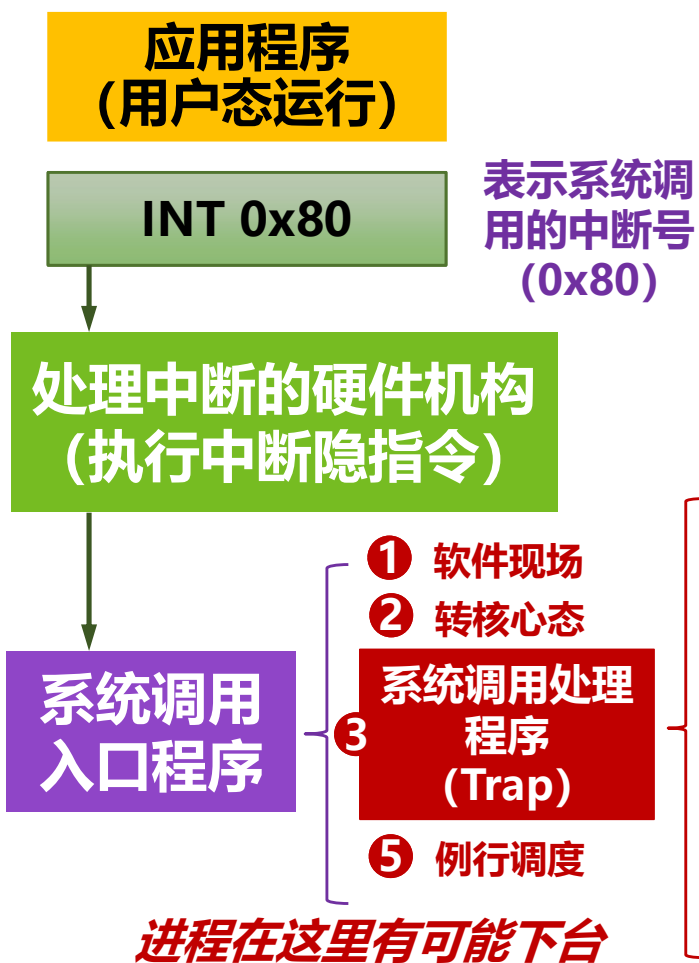




UNIX系统调用

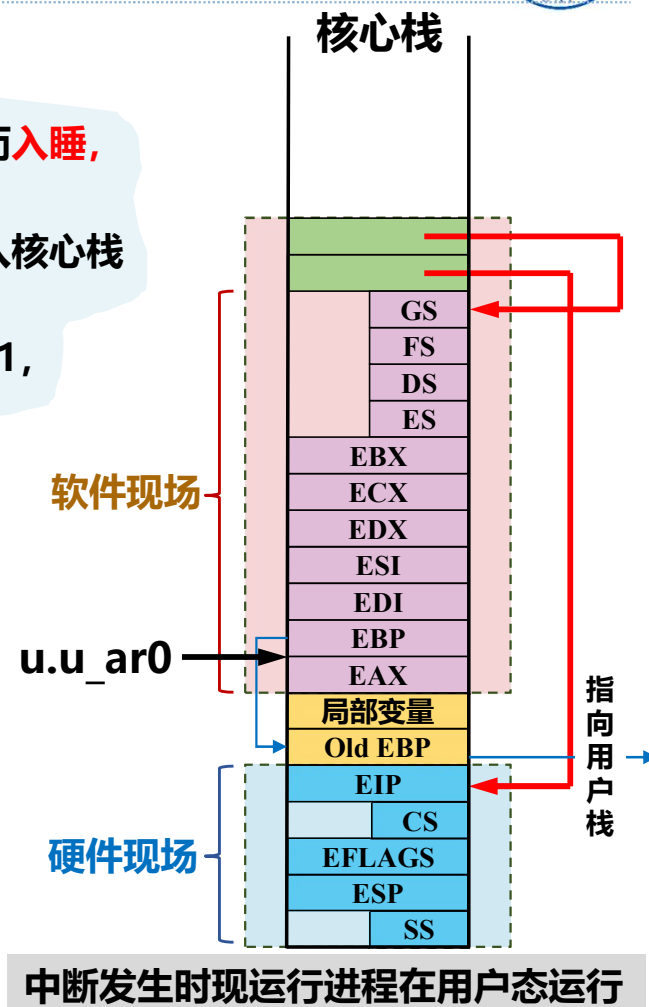
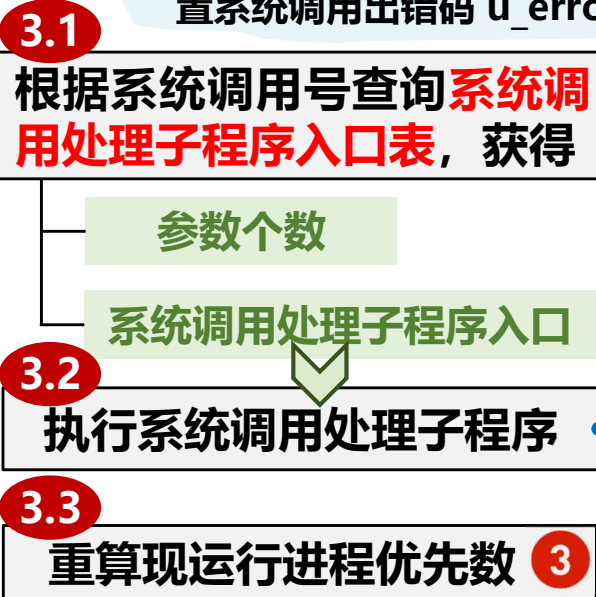


系统调用详细流程



功能虽不同, 但遵循以下规则:

1. 参数取自user结构;
2. 进程可能会因为使用系统资源而**入睡**, **下台**, 等待被唤醒后重新上台;
3. 如果系统调用成功, 返回值填入核心栈中保存EAX的位置;
如果系统调用失败, 该位置 = 1, 置系统调用出错码 u_error

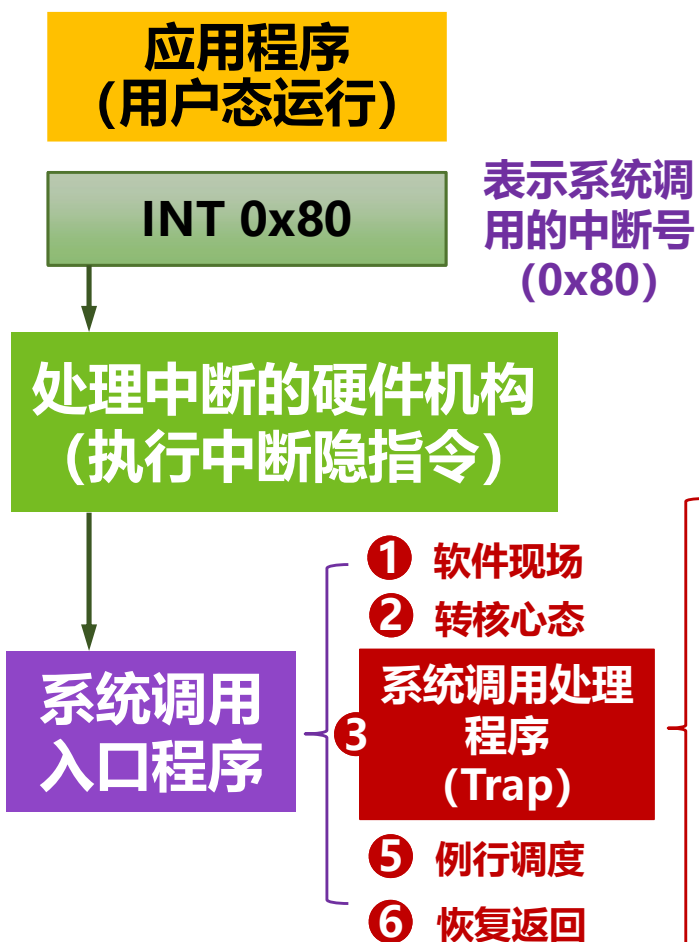




UNIX系统调用

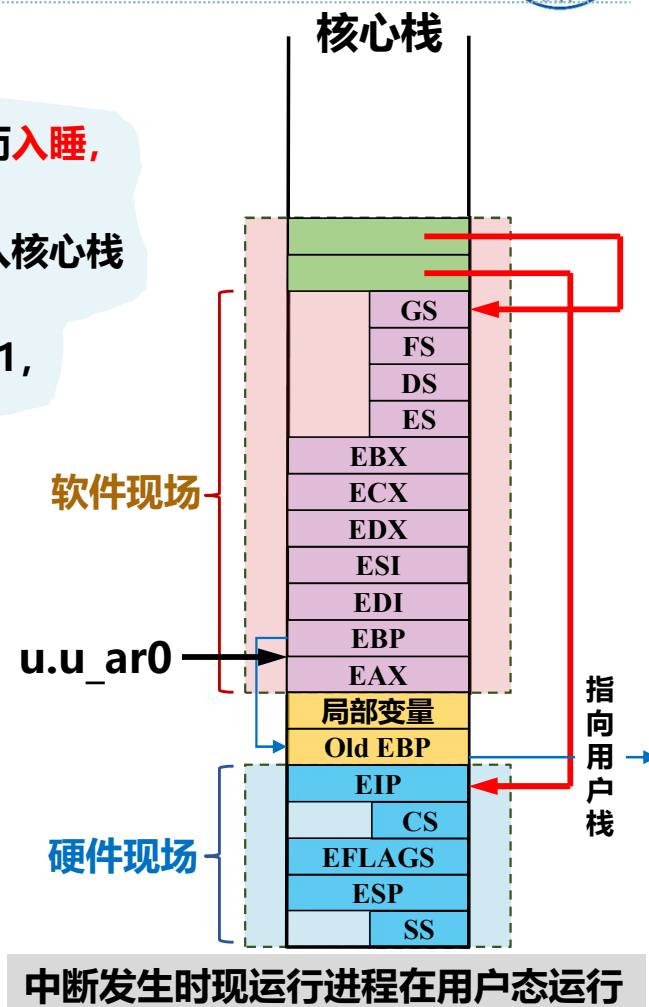
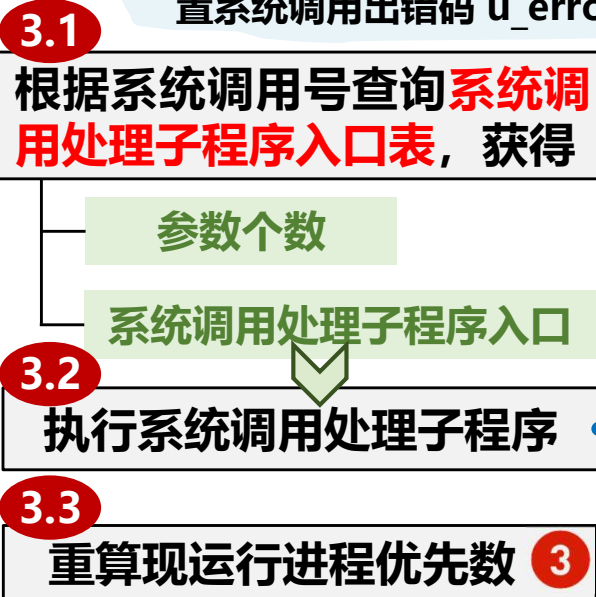


系统调用详细流程



功能虽不同，但遵循以下规则：

1. 参数取自user结构；
2. 进程可能会因为使用系统资源而**入睡**，**下台**，等待被唤醒后重新上台；
3. 如果系统调用成功，返回值填入核心栈中保存EAX的位置；如果系统调用失败，该位置 = 1，置系统调用出错码 u_error

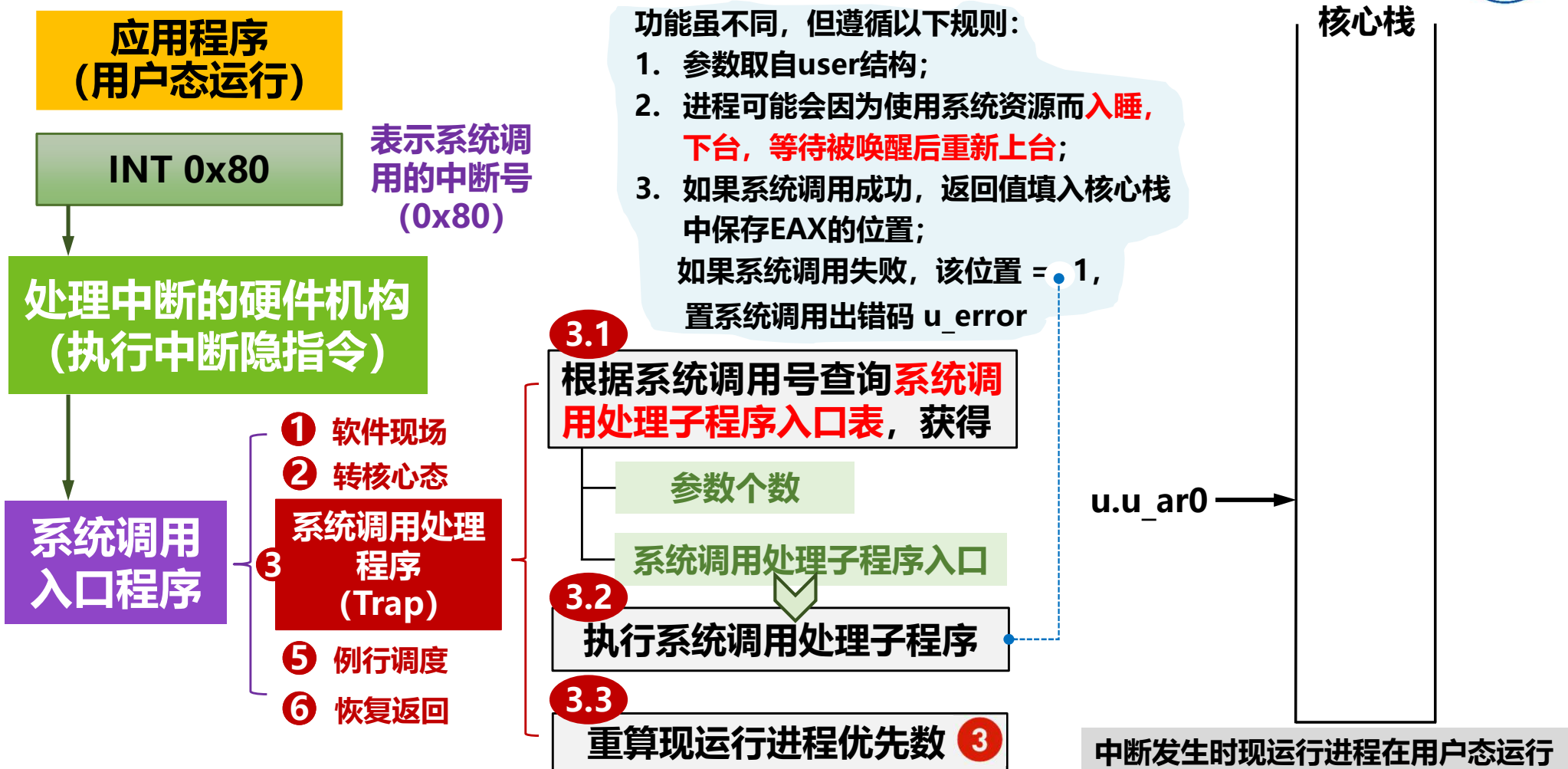




UNIX系统调用



系统调用详细流程

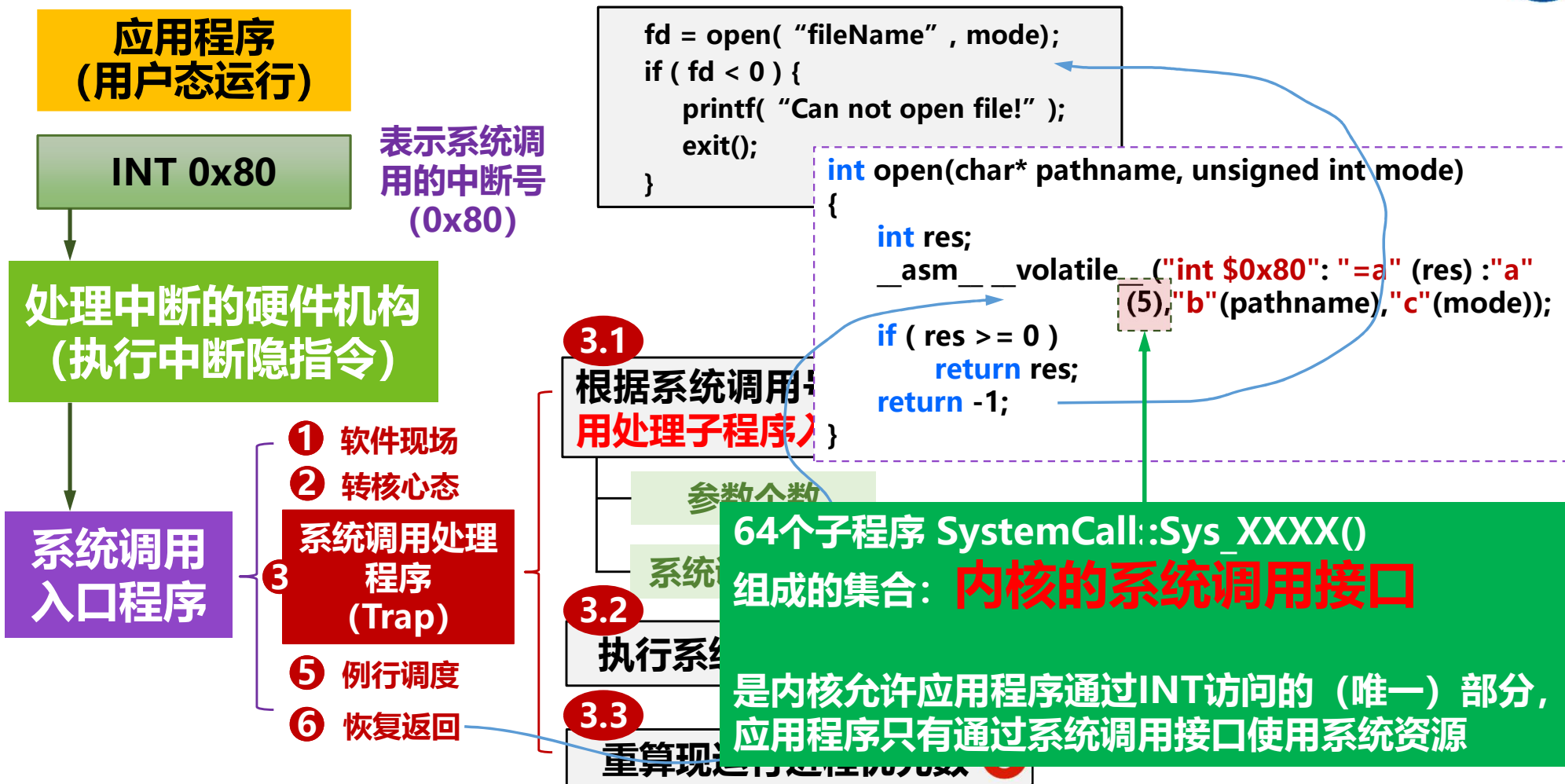




UNIX系统调用



系统调用详细流程





UNIX系统调用



系统调用和一般函数调用的区别

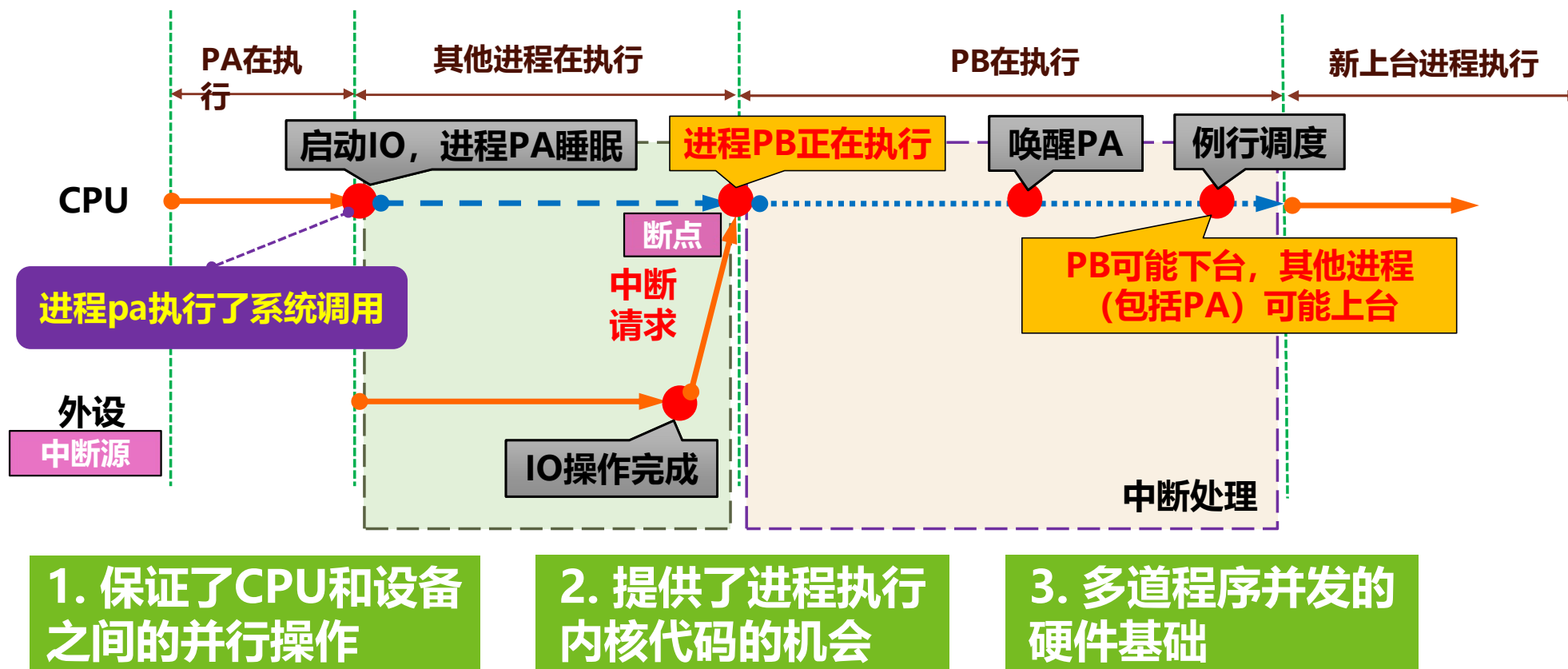
项目	一般子程序调用	系统调用
调用者	应用程序或内核中的任意子程序	应用程序中的某个子程序
被调用者	同一个程序中的另一个子程序	操作系统内部供应用程序调用的子程序
调用者和被调用者之间的关系	链接在同一个可执行文件中	分别链接在两个不同的可执行文件中，被调用者一定是操作系统内核
引发调用的事件	call指令	INT 指令
被调用者栈帧位置	当前堆栈的栈顶	核心栈
参数如何传递	调用者向当前堆栈push实参	调用者和被调用者的栈帧一定分别位于同一个进程的用户栈和核心栈。参数传递需借助寄存器和user结构
调用者如何得到返回值	CPU内的寄存器	将系统调用的返回值写入核心栈底保存EAX寄存器的位置，待中断返回用户态后该值将pop回该寄存器，作为系统调用的返回值



UNIX系统调用



系统调用与中断的关系

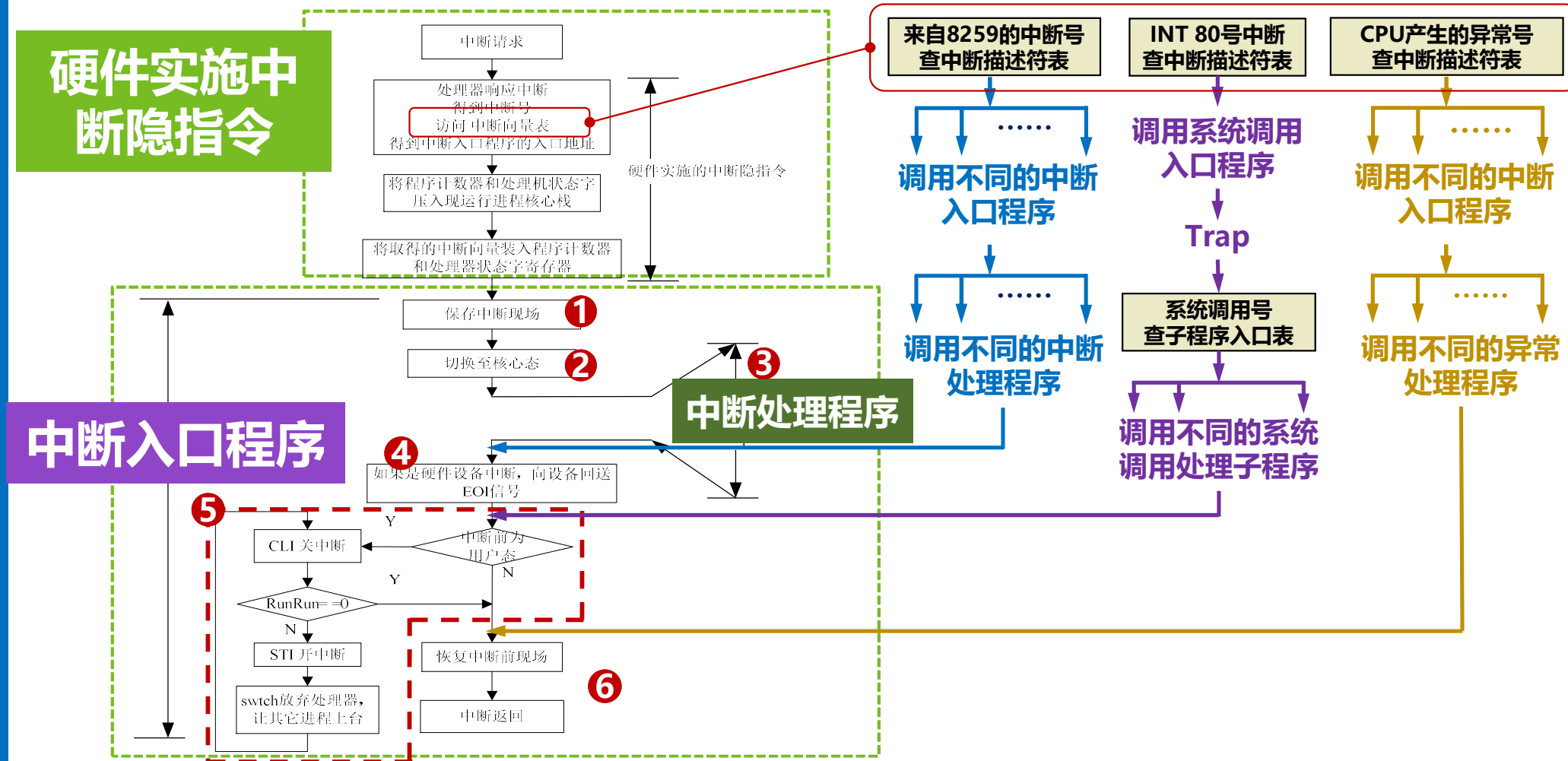




总结



关于中断





本节小结



1 中断的基本概念

2 UNIX中断的处理过程

阅读教材：103页 ~ 110页